

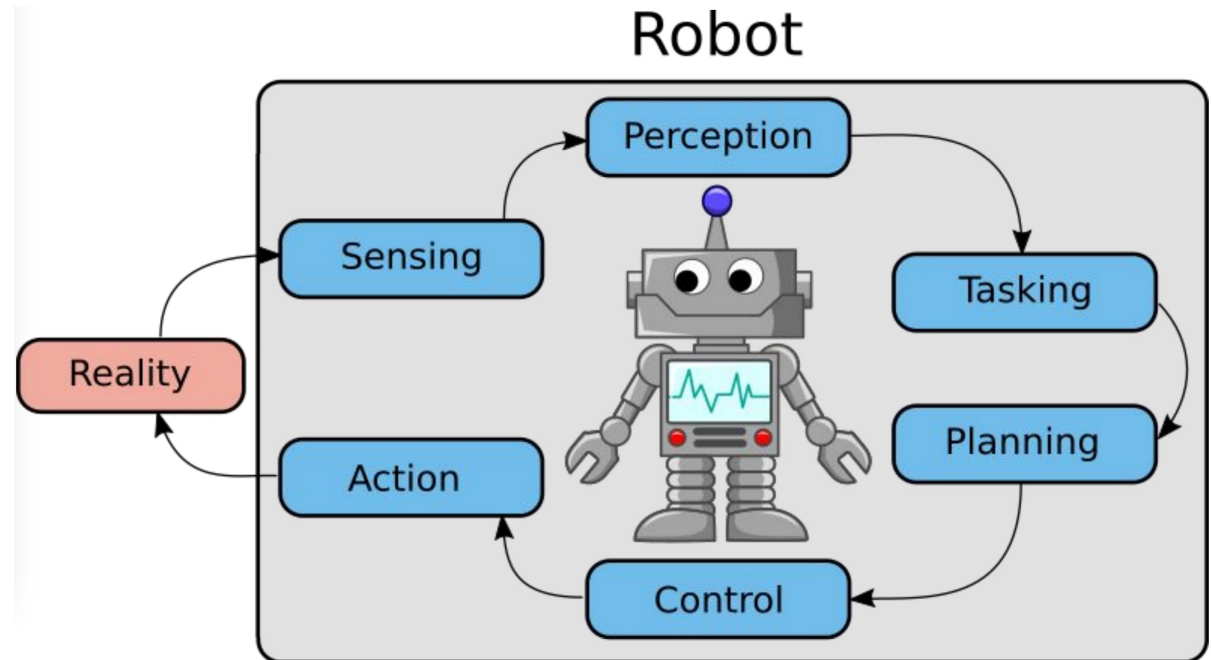
Robot Learning

Lecture 2a. Kinematics of Robot Manipulators

What is robotics?

- From "*The intelligent connection of perception to action*", J. M. Bradley, MIT AI Lab, 1986

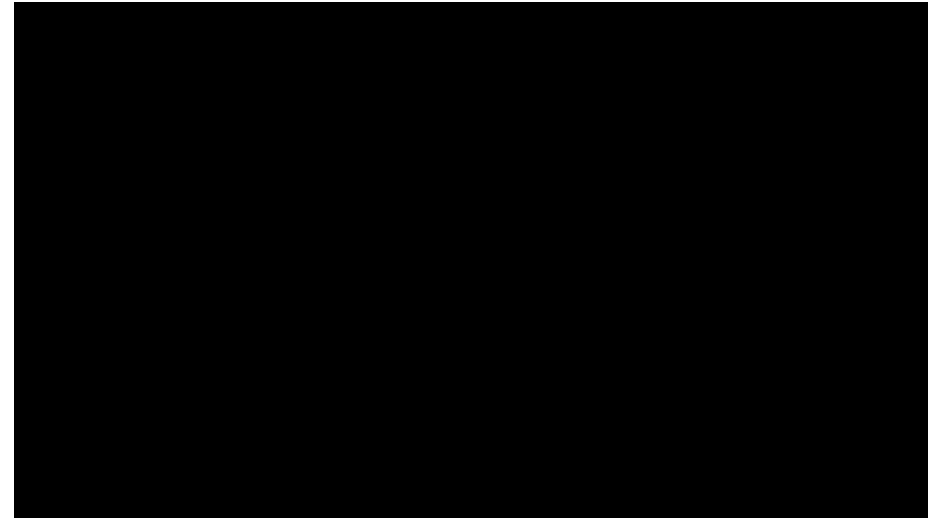
*"Robotics is the **intelligent** connection of the **perception to action**"*



Action implies motion



Work from MIT Media Lab, ICRA 2021



Demo from META at CVPR 2023

Action implies motion



Champion-Level Performance in Drone Racing using Deep Reinforcement Learning

E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza

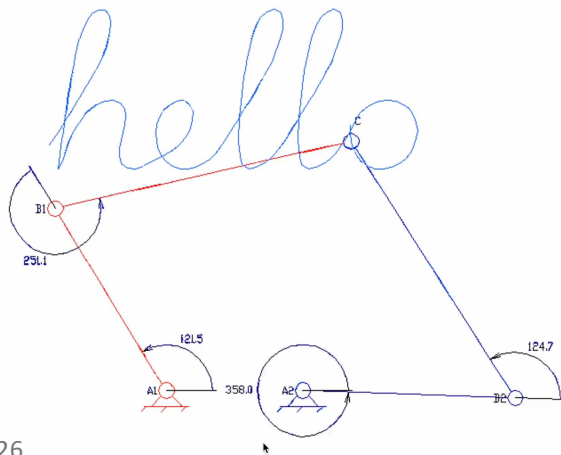


University of
Zurich^{UZH}



Kinematics

- **Kinematics** studies the geometric and timing aspects of robot motion, without reference to the causes producing it
- We can see a robot as a kinematic chain of rigid bodies interconnected by joints, and we want to figure out the relation between the motion of the individual joints and the overall motion of the robot.



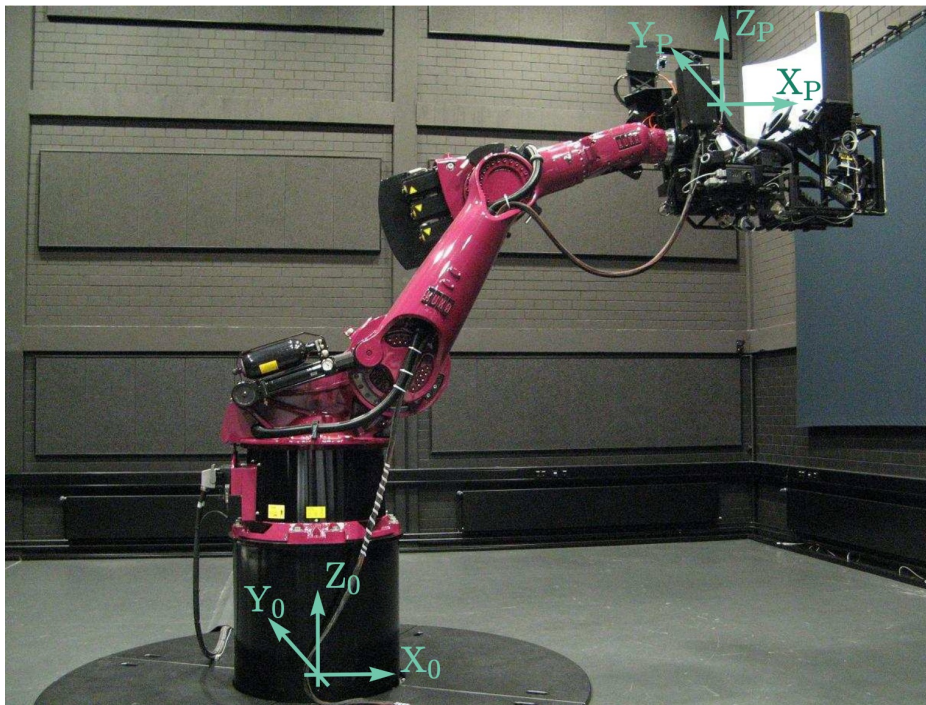
A.Y. 2025-2026



Robot Learning

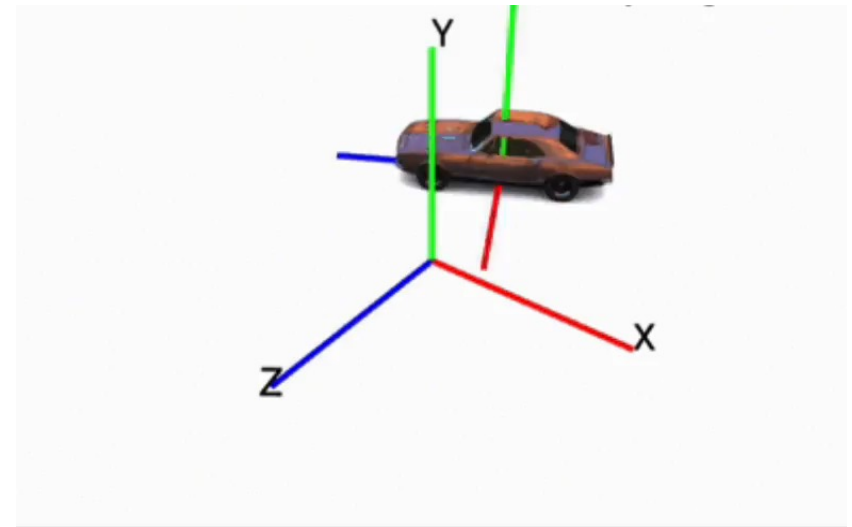
Kinematics

- From one coordinate frame (the world frame) to the one that represents the robot (end-effector, vehicle, ...)

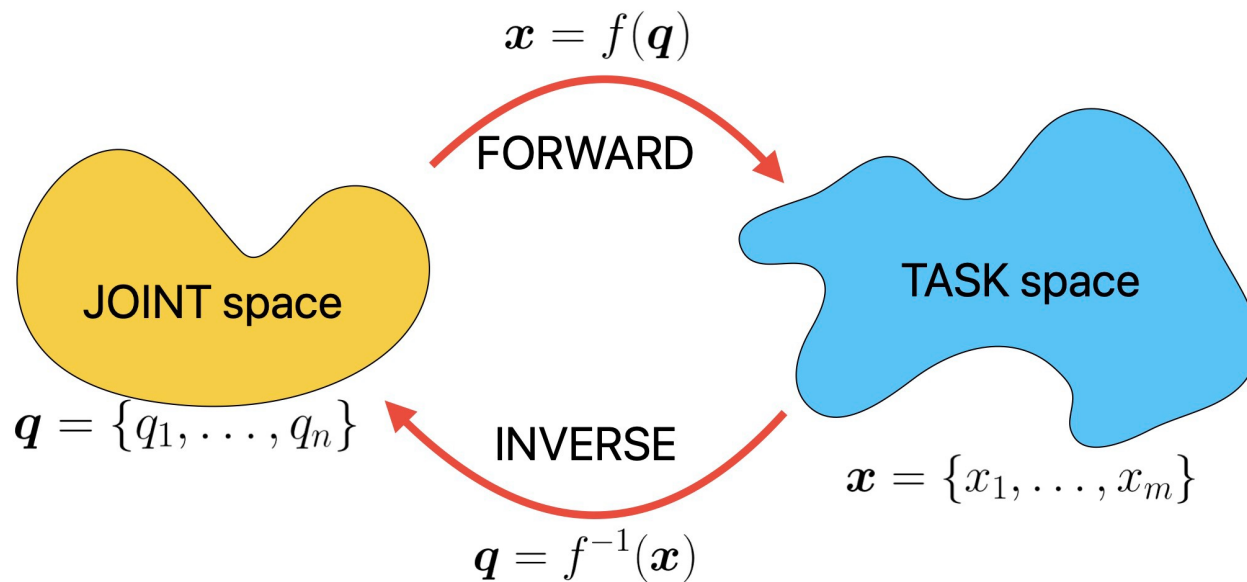


A.Y. 2025-2026

Robot Learning



Definitions



- ***parameterization q***

- unambiguous and minimal characterization of robot configuration
- n = #degrees of freedom (or robot joints)

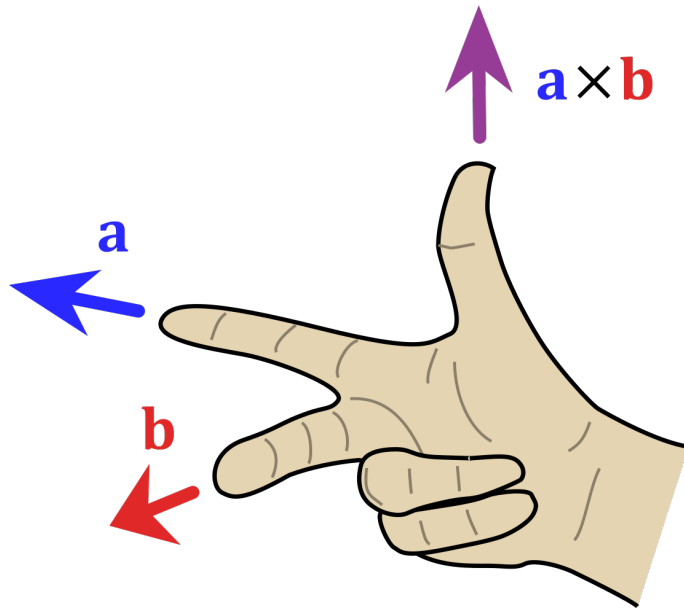
- ***parameterization x***

- description of pose of interest to the required task
- usually $m \leq n$ (but not always)

Position and orientation representation

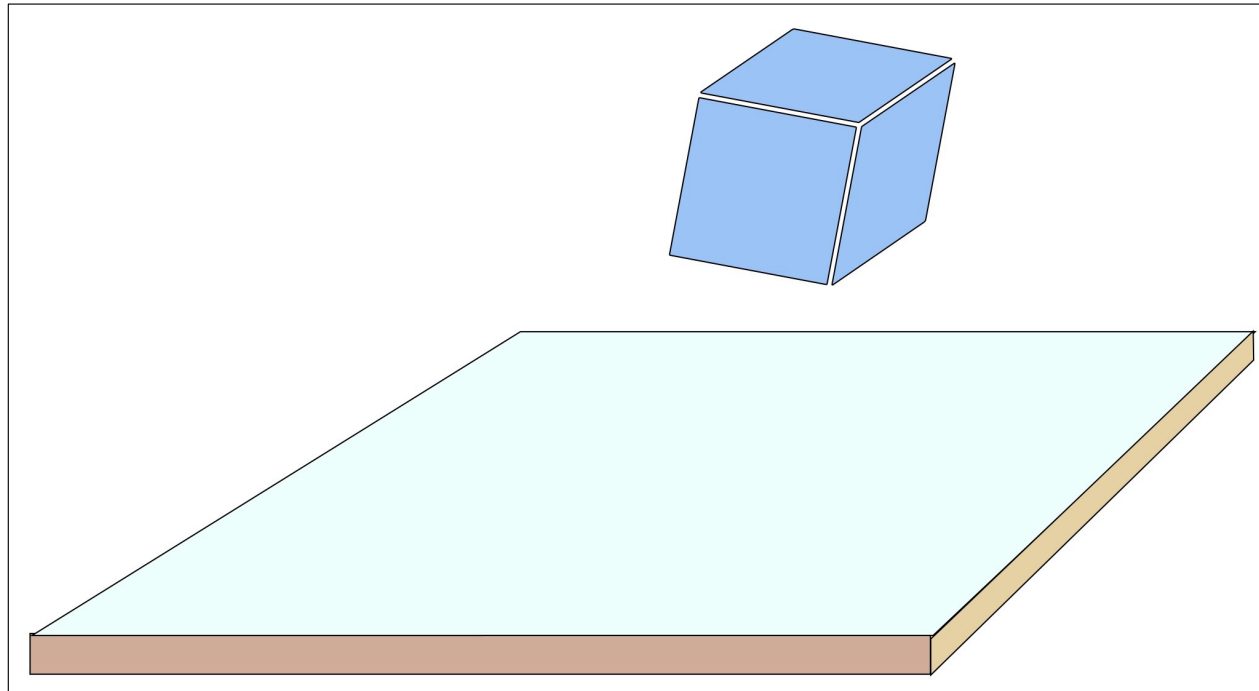
Right hand rule

- First and foremost, do you remember the right-hand rule?



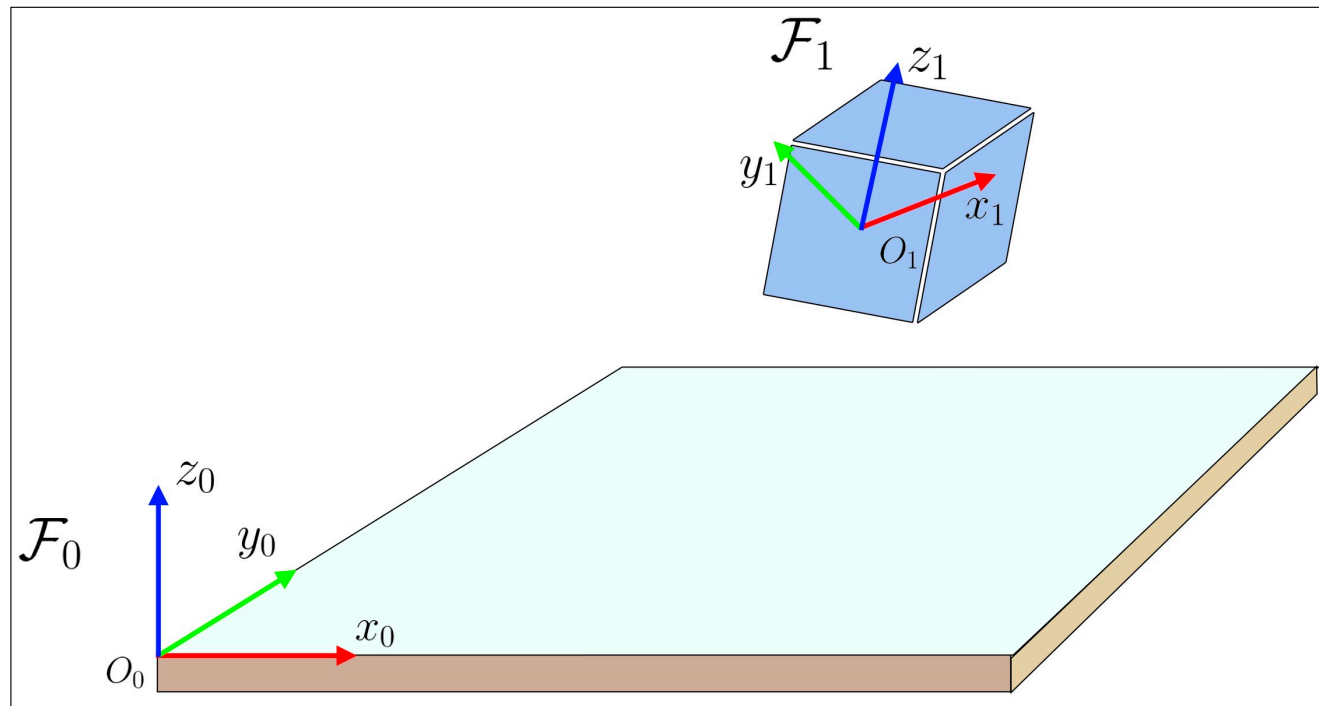
Position and Orientation of a Rigid Body

- Given a rigid body in space, how can we characterize its position and orientation in space?



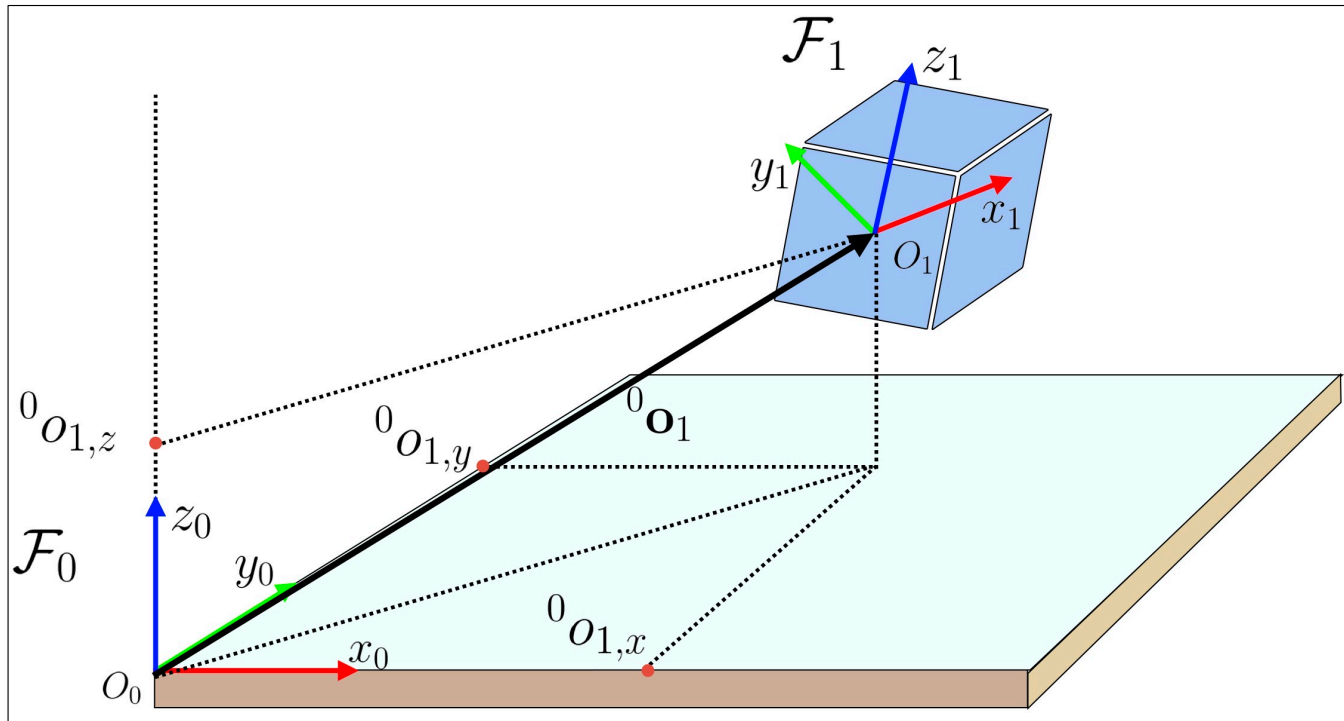
Position and Orientation of a Rigid Body

- We do it by considering a reference frame fixed in the world, and a second reference frame attached to the rigid body.



Position

- The position of the rigid body is represented by the vector from O_0 to O_1 , expressed in F_0 .

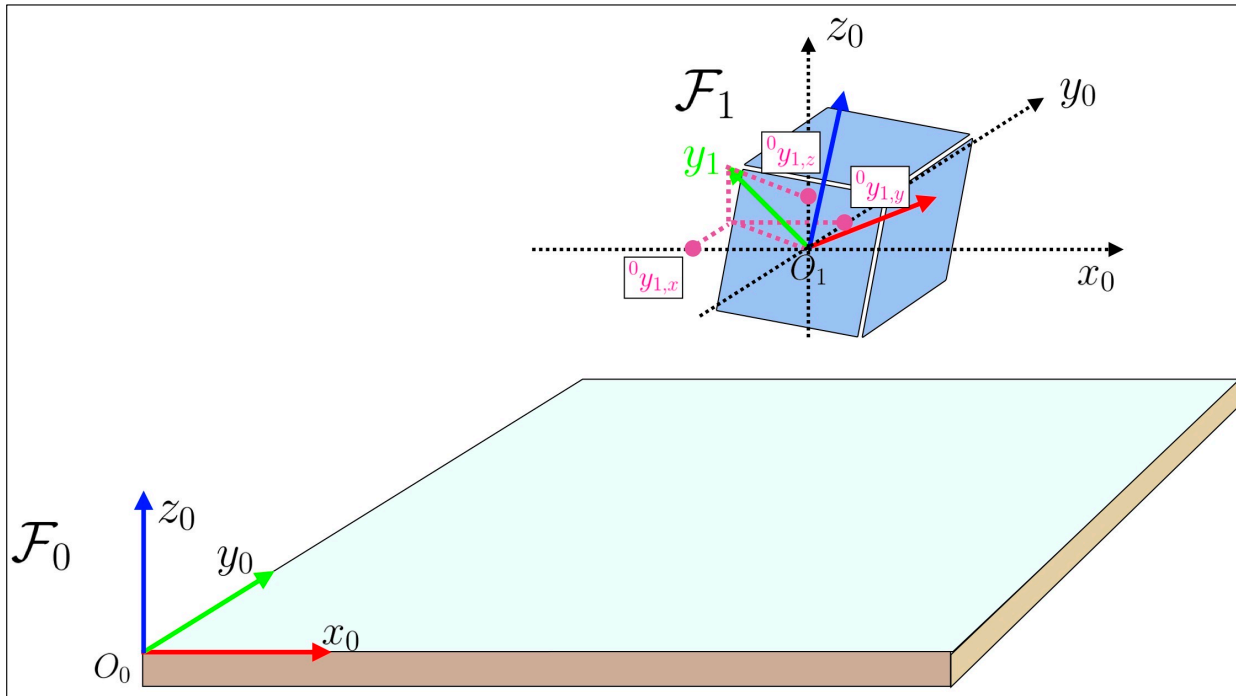


The **position vector** is

$${}^0\mathbf{o}_1 = \begin{bmatrix} {}^0O_{1,x} \\ {}^0O_{1,y} \\ {}^0O_{1,z} \end{bmatrix}$$

Orientation

- The representation of the orientation can be made considering unit length vectors along the axes of the rotated frame and evaluating their components in the reference frame:



The **unit vector** that represents the axis y_1 in F_0 is

$${}^0\mathbf{y}_1 = \begin{bmatrix} {}^0y_{1,x} \\ {}^0y_{1,y} \\ {}^0y_{1,z} \end{bmatrix}$$

Similar expressions hold for ${}^0\mathbf{x}_1$ and ${}^0\mathbf{z}_1$

Orientation

- We can group the unit vectors ${}^0\mathbf{x}_1$, ${}^0\mathbf{y}_1$ and ${}^0\mathbf{z}_1$ in a matrix: the **rotation matrix** 0R_1

$${}^0R_1 = \begin{bmatrix} {}^0x_{1,x} & {}^0y_{1,x} & {}^0z_{1,x} \\ {}^0x_{1,y} & {}^0y_{1,y} & {}^0z_{1,y} \\ {}^0x_{1,z} & {}^0y_{1,z} & {}^0z_{1,z} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1^T \mathbf{x}_0 & \mathbf{y}_1^T \mathbf{x}_0 & \mathbf{z}_1^T \mathbf{x}_0 \\ \mathbf{x}_1^T \mathbf{y}_0 & \mathbf{y}_1^T \mathbf{y}_0 & \mathbf{z}_1^T \mathbf{y}_0 \\ \mathbf{x}_1^T \mathbf{z}_0 & \mathbf{y}_1^T \mathbf{z}_0 & \mathbf{z}_1^T \mathbf{z}_0 \end{bmatrix}$$

$\uparrow$
 ${}^0\mathbf{x}_1$

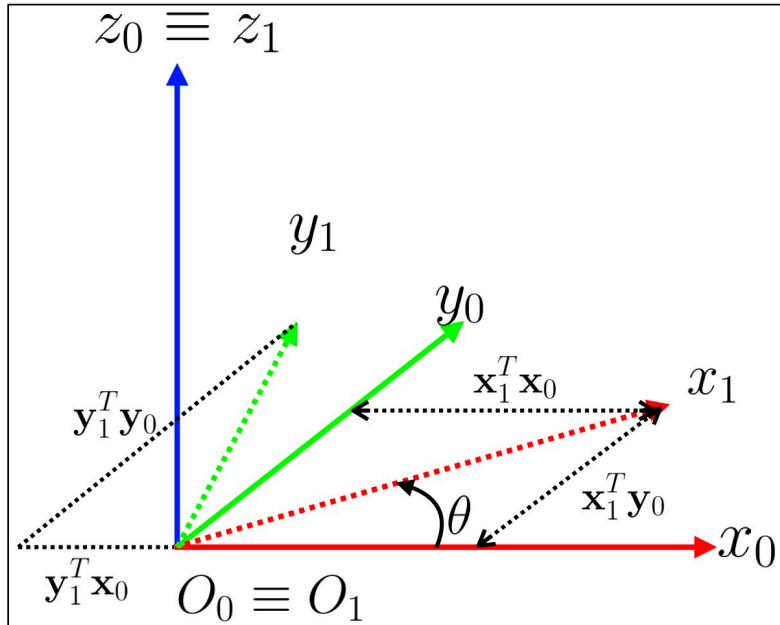
$\uparrow$
 ${}^0\mathbf{y}_1$

$\uparrow$
 ${}^0\mathbf{z}_1$

Rotation Matrices

- Rotation matrices are **orthonormal**:
 - Their **columns** are **orthonormal** vectors: they are **orthogonal** and have **unitary norm**.
 - $\det(R)=1$. This makes rotation matrices a special subset of orthonormal matrices (known **Special Orthogonal Group $SO(3)$**)
 - $R^{-1} = R^T$

Elementary Rotation Matrices



$$\begin{aligned}x_1^T x_0 &= \cos(\theta), & y_1^T x_0 &= -\sin(\theta) \\x_1^T y_0 &= \sin(\theta), & y_1^T y_0 &= \cos(\theta) \\z_1^T z_0 &= 1\end{aligned}$$



$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

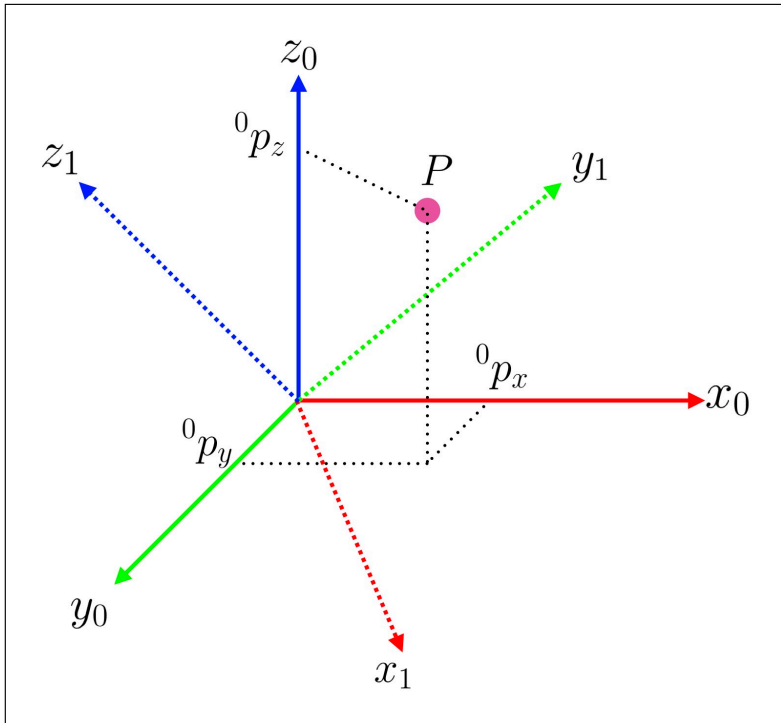
Similarly for X and Y:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

Change of coordinates of a point

Let's consider now a point P seen from two different frames of reference

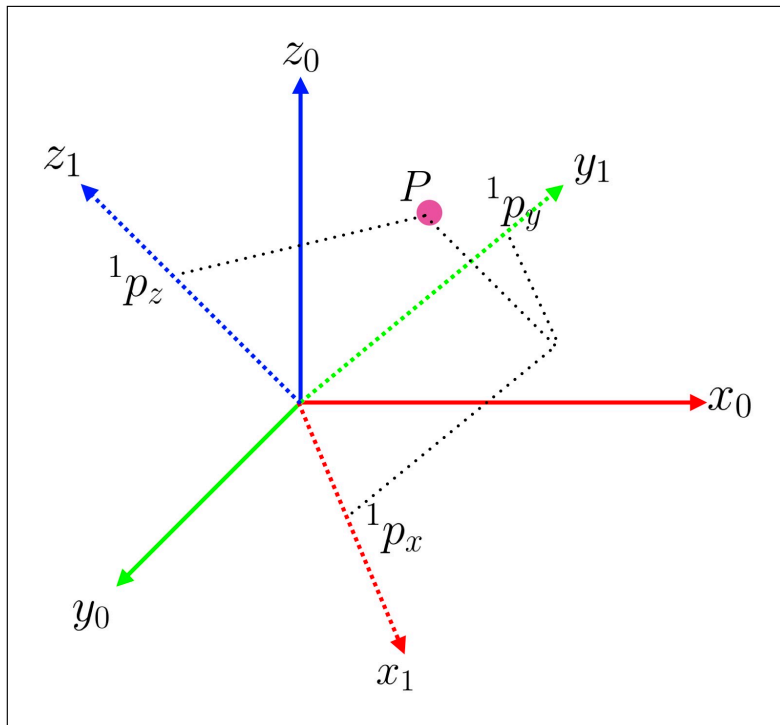


The coordinates of the **same point** in the two frames of reference are:

$${}^0\mathbf{p} = \begin{bmatrix} {}^0p_x \\ {}^0p_y \\ {}^0p_z \end{bmatrix}$$

Change of coordinates of a point

Let's consider now a point P seen from two different frames of reference



The coordinates of the **same point** in the two frames of reference are:

$${}^0\mathbf{p} = \begin{bmatrix} {}^0p_x \\ {}^0p_y \\ {}^0p_z \end{bmatrix} \quad {}^1\mathbf{p} = \begin{bmatrix} {}^1p_x \\ {}^1p_y \\ {}^1p_z \end{bmatrix}$$

Therefore:

$$\begin{aligned} {}^0\mathbf{p} &= {}^0\mathbf{x}_1 {}^1p_x + {}^0\mathbf{y}_1 {}^1p_y + {}^0\mathbf{z}_1 {}^1p_z \\ &= \begin{bmatrix} {}^0\mathbf{x}_1 & {}^0\mathbf{y}_1 & {}^0\mathbf{z}_1 \end{bmatrix} {}^1\mathbf{p} = {}^0R_1 {}^1\mathbf{p} \end{aligned}$$

The matrix 0R_1 encodes the **change of coordinates** of a point, from the frame F_1 to the frame F_0

Change of coordinates of a point

Assume now that we have three frames of reference (with the same origin): F_0, F_1, F_2 .

- The change of coordinates from F_1 to F_0 is

$${}^0p = {}^0R_1 {}^1p$$

- The change of coordinates from F_2 to F_1 is

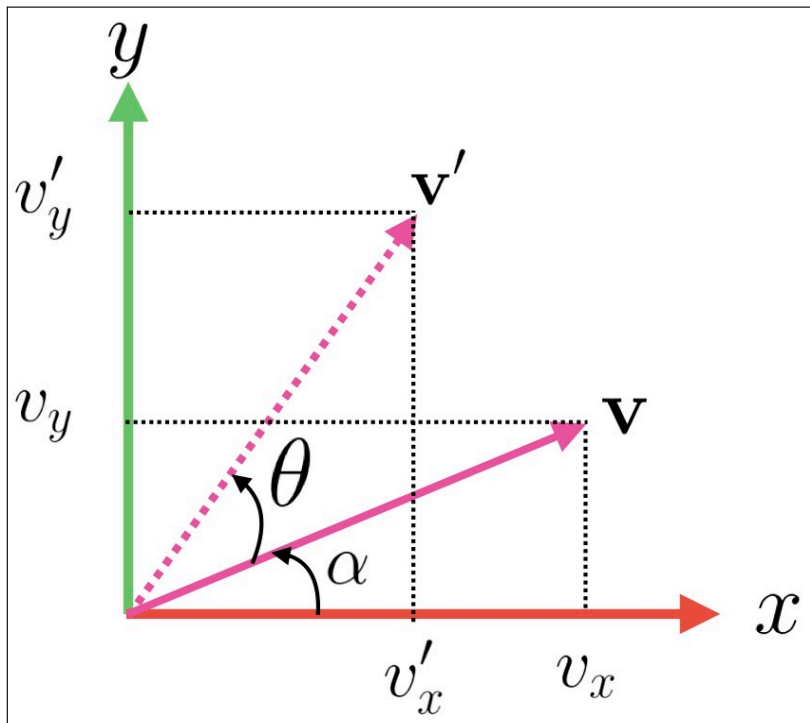
$${}^1p = {}^1R_2 {}^2p$$

$${}^0p = {}^0R_1 {}^1R_2 {}^2p$$

Chain rule property: ${}^0R_1 \cdot {}^1R_2 = {}^0R_2$

Rotation of a Vector

Let's consider a vector $\mathbf{v}$ in a frame of reference, and rotate the vector



$$v_x = |\mathbf{v}| \cos(\alpha)$$

The components of $\mathbf{v}$ are: $v_y = |\mathbf{v}| \sin(\alpha)$

$$v_z$$

The components of $\mathbf{v}'$ are:

$$v'_x = |\mathbf{v}| \cos(\alpha + \theta)$$

$$= |\mathbf{v}| \cos(\alpha) \cdot \cos(\theta) - |\mathbf{v}| \sin(\alpha) \cdot \sin(\theta)$$

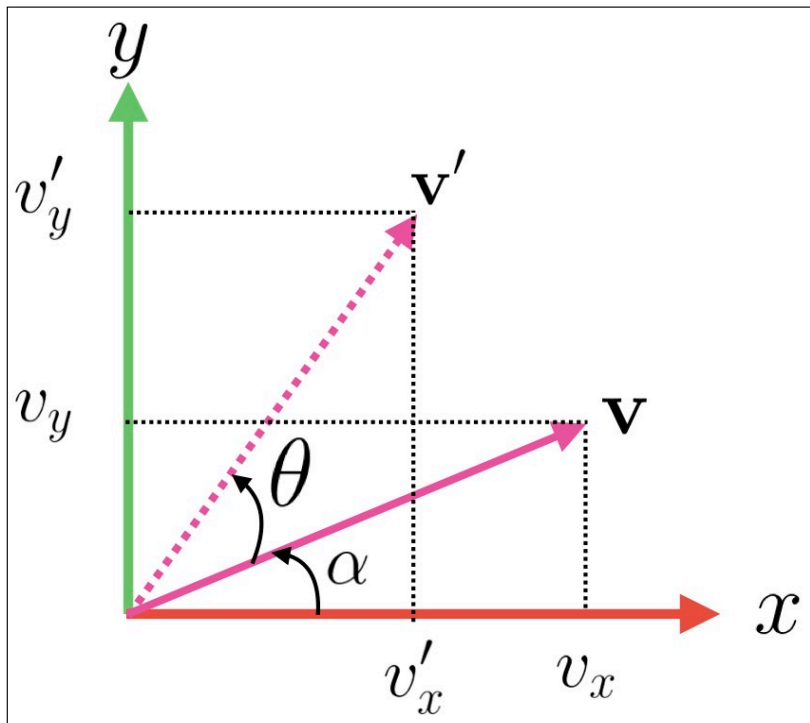
$$v'_y = |\mathbf{v}| \sin(\alpha + \theta)$$

$$= |\mathbf{v}| \sin(\alpha) \cdot \cos(\theta) + |\mathbf{v}| \cos(\alpha) \cdot \sin(\theta)$$

$$v'_z = v_z$$

Rotation of a Vector

Let's consider a vector $\mathbf{v}$ in a frame of reference, and rotate the vector



$$v_x = |\mathbf{v}| \cos(\alpha)$$

The components of $\mathbf{v}$ are:

$$v_y = |\mathbf{v}| \sin(\alpha)$$

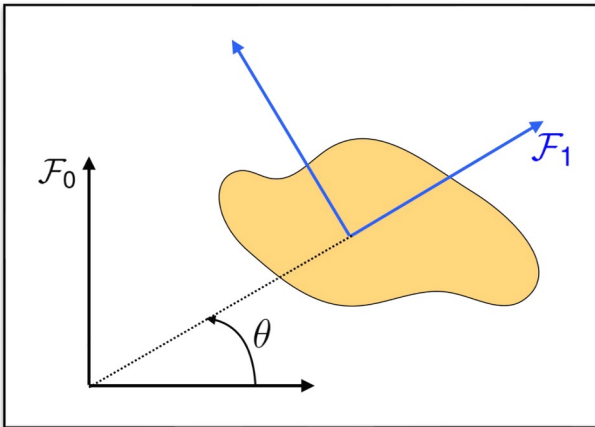
$$v_z$$

The components of $\mathbf{v}'$ are:

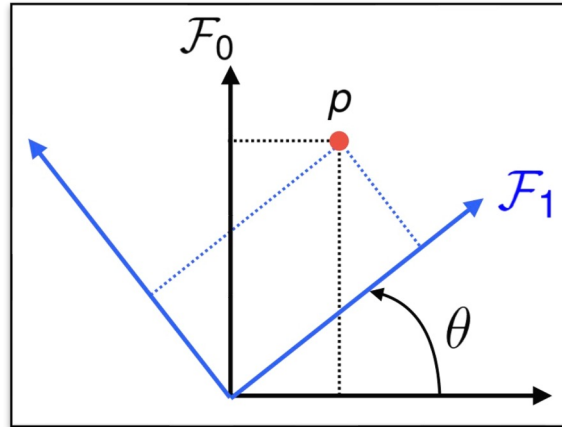
$$\mathbf{v}' = \begin{bmatrix} c\theta & -s\theta & 0 \\ s\theta & c\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \mathbf{v} = R_z(\theta) \mathbf{v}$$

Interpretations of a Rotation Matrix

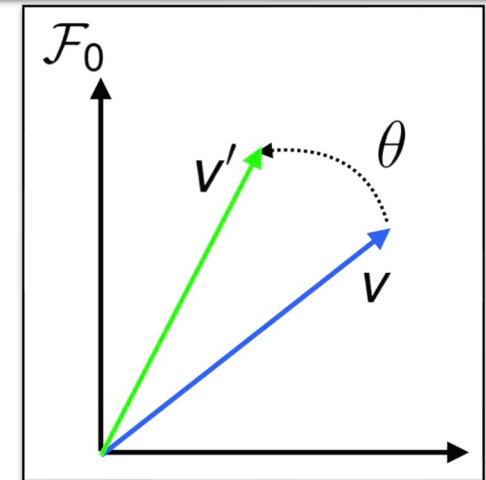
The rotation matrix ${}^0R_1(\theta)$ is an operator that superposes frame F_0 to frame F_1 .



The **orientation of a rigid body** w.r.t. a reference frame F_0



The **change of coordinates of a point p** from frame F_1 to frame F_0



The **rotation of a vector** in the same frame of reference

Representations of orientation

- A rotation matrix represents the orientation of a frame with respect to another one by means of **9 parameters**, among which 6 constraints exist.

$${}^0R_1 = \begin{bmatrix} {}^0x_{1,x} & {}^0y_{1,x} & {}^0z_{1,x} \\ {}^0x_{1,y} & {}^0y_{1,y} & {}^0z_{1,y} \\ {}^0x_{1,z} & {}^0y_{1,z} & {}^0z_{1,z} \end{bmatrix} = \begin{bmatrix} {}^0\mathbf{x}_1 & {}^0\mathbf{y}_1 & {}^0\mathbf{z}_1 \end{bmatrix} \quad \boxed{\text{9 elements}}$$

- **Unitary norm** constraints (3): $\|{}^0\mathbf{x}_1\| = 1, \|{}^0\mathbf{y}_1\| = 1, \|{}^0\mathbf{z}_1\| = 1$
- **Orthogonality** constraints (3): ${}^0\mathbf{x}_1^T {}^0\mathbf{y}_1 = 0, {}^0\mathbf{x}_1^T {}^0\mathbf{z}_1 = 0, {}^0\mathbf{y}_1^T {}^0\mathbf{z}_1 = 0$



A minimal representation of the orientation has 3 parameters

Representations of orientation

Possible representations of orientations are:

- **Rotation matrix** [9 parameters]
- Sequence of 3 elementary rotations [3 parameters]
- **Axis/angle** (4 parameters)
- **Quaternions** (4 parameters)

w.r.t. current frame
(**Euler Angles**)

w.r.t. fixed frame
(**Fixed Angles**)

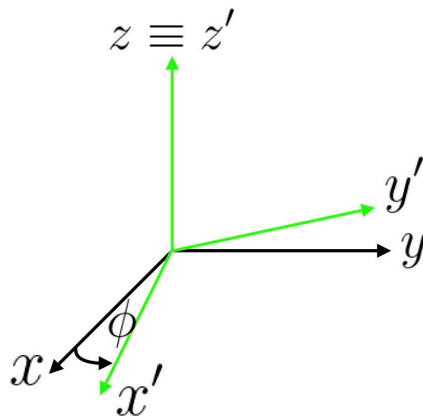
Euler Angles

We can represent an arbitrary orientation as a sequence of 3 elementary rotations.

- If the elementary rotations are expressed w.r.t. the current frame, they are called **Euler Angles**

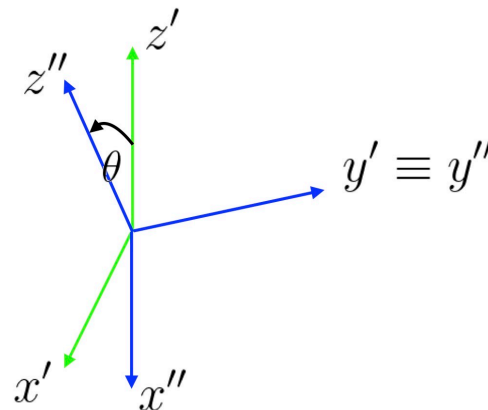
Example: ZYZ Euler Angles

Rotation around Z



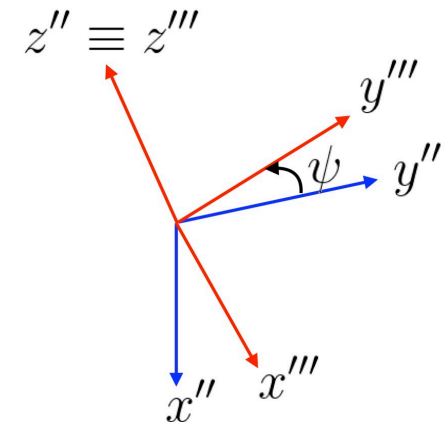
A.Y. 2025-2026

Rotation around Y



Robot Learning

Rotation around Z



25

Euler Angles

Example: ZYZ Euler Angles

Rotation around Z

$$R_z(\phi) = \begin{bmatrix} c\phi & -s\phi & 0 \\ s\phi & c\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Rotation around Y

$$R_{y'}(\theta) = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}$$

Rotation around Z

$$R_{z''}(\psi) = \begin{bmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

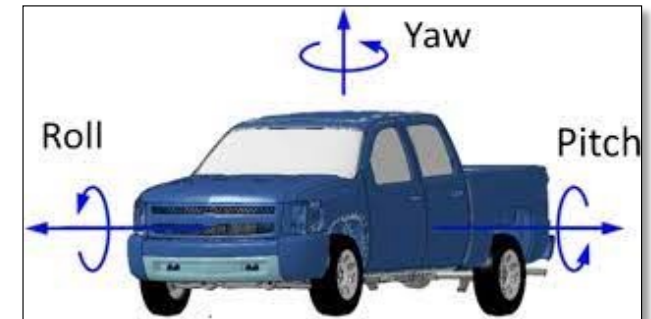
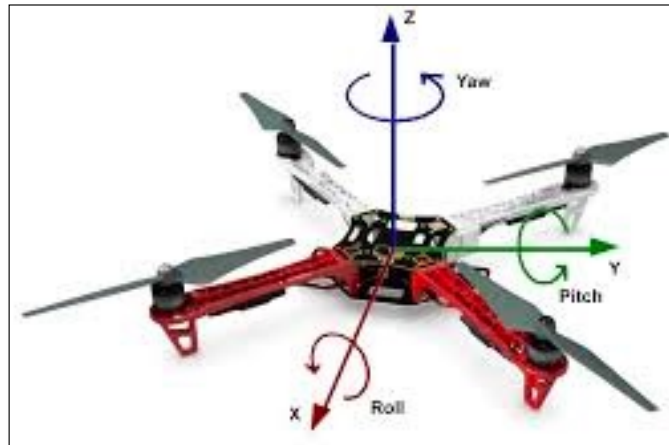
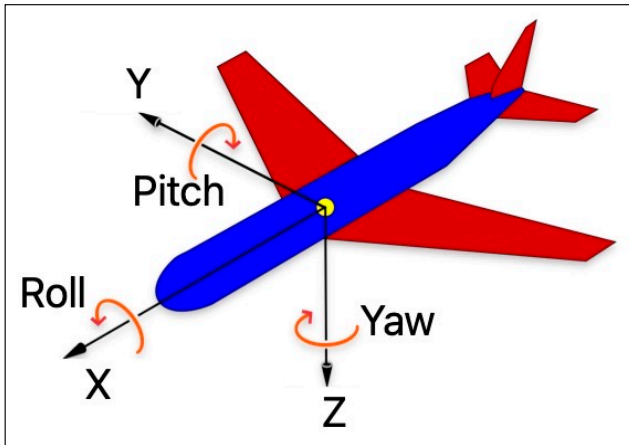
$$R = R_z(\phi) R_{y'}(\theta) R_{z''}(\psi) = \begin{bmatrix} c\phi c\theta c\psi - s\phi s\psi & -c\phi c\theta s\psi - s\phi c\psi & c\phi s\theta \\ s\phi c\theta c\psi + c\phi s\psi & -s\phi c\theta s\psi + c\phi c\psi & s\phi s\theta \\ -s\theta c\psi & s\theta s\psi & c\theta \end{bmatrix}$$

With **Euler Angles** the elementary rotation matrices are multiplied **left-to-right** in the order of the sequence

Euler Angles Angles

Example: Roll-Pitch-Yaw (ZYX Euler Angles)

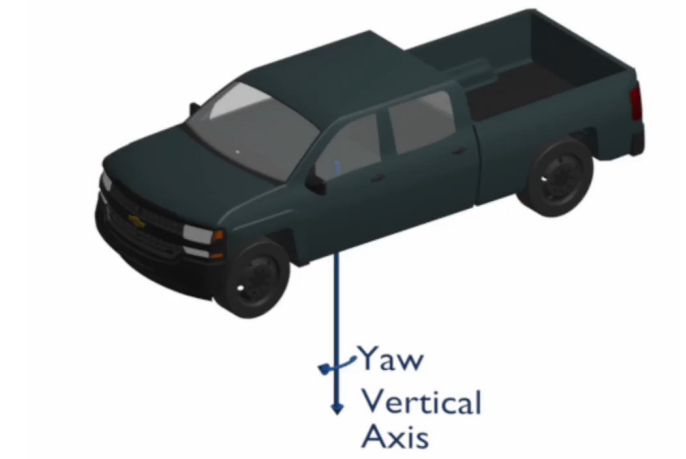
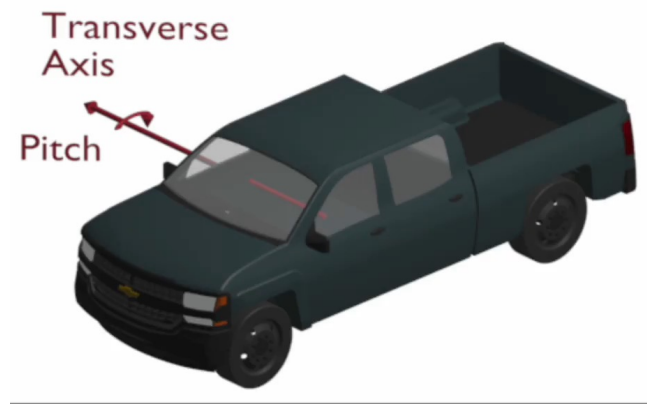
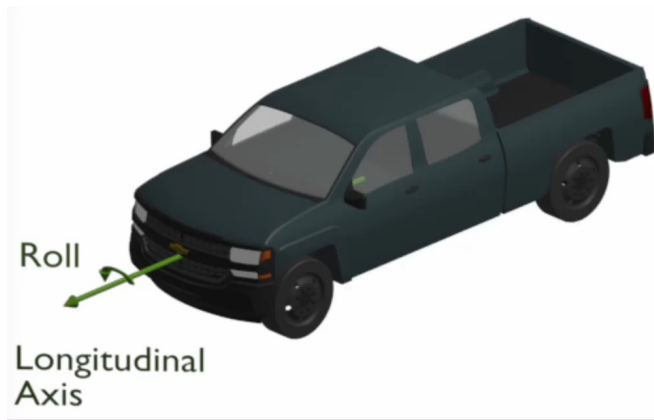
Notation derived from aeronautics, but commonly used for mobile robots in general.



Euler Angles Angles

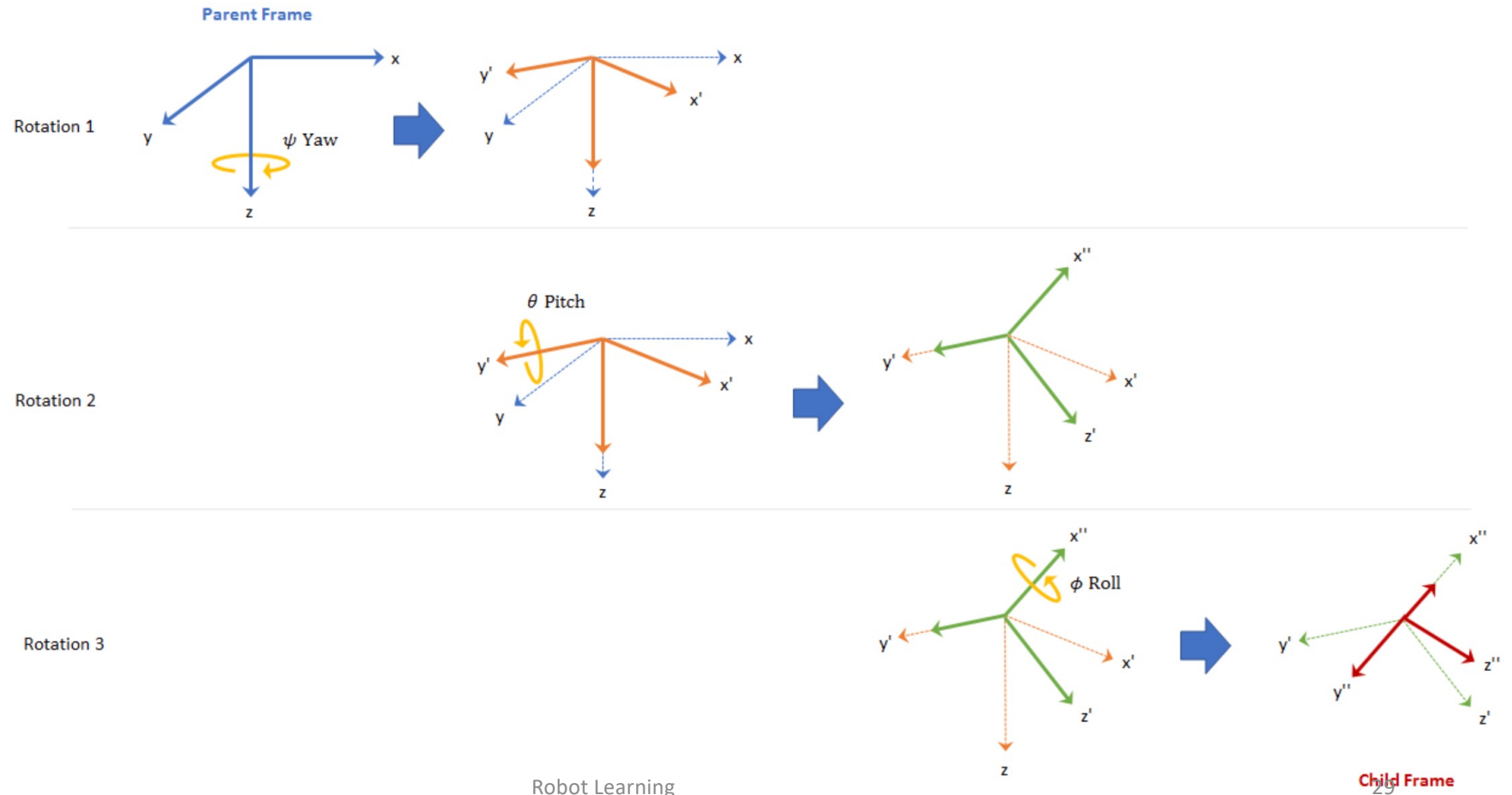
Example: Roll-Pitch-Yaw (ZYX Euler Angles)

Notation derived from aeronautics, but commonly used for mobile robots in general.



Euler Angles

Example: Roll-
Pitch-Yaw (ZYX
Euler Angles)



Euler Angles

Example: Roll-Pitch-Yaw (ZYX Euler Angles)

Rotation around X

$$R_x(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\psi & -s\psi \\ 0 & s\psi & c\psi \end{bmatrix}$$

Rotation around Y

$$R_y(\theta) = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}$$

Rotation around Z

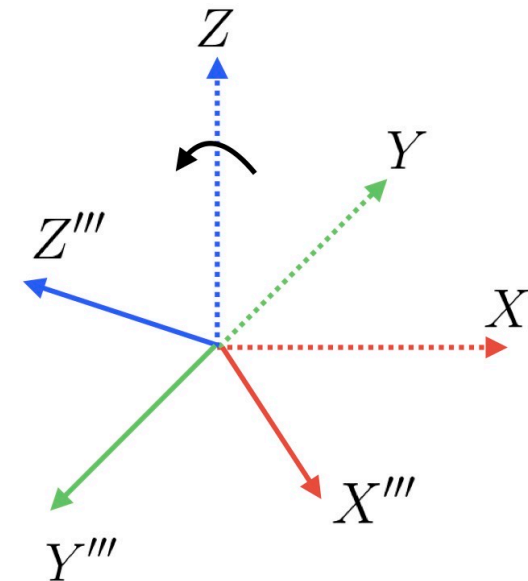
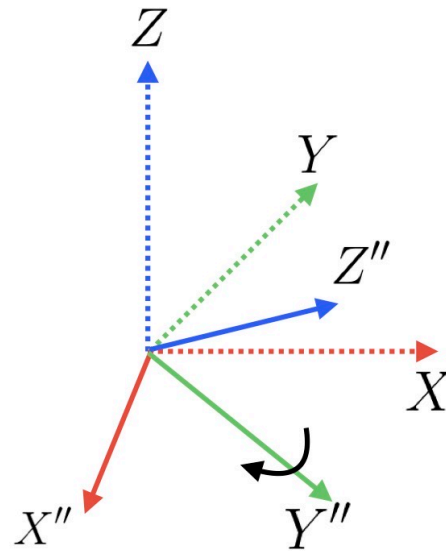
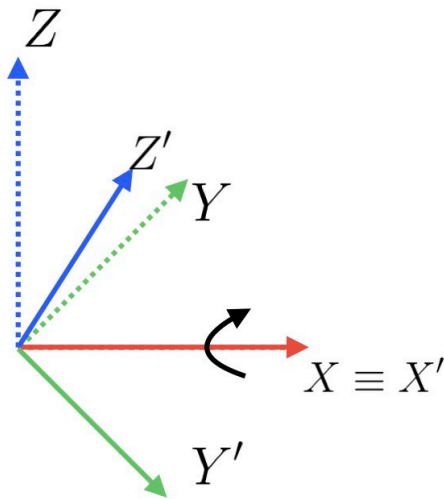
$$R_z(\phi) = \begin{bmatrix} c\phi & -s\phi & 0 \\ s\phi & c\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R = R_z(\phi) R_y(\theta) R_x(\psi) = \begin{bmatrix} c\phi c\theta & -s\phi c\psi + c\phi s\theta s\psi & s\phi s\psi + c\phi s\theta c\psi \\ s\phi c\theta & c\phi c\psi + s\phi s\theta s\psi & -c\phi s\psi + s\phi s\theta c\psi \\ -s\theta & c\theta s\psi & c\theta c\psi \end{bmatrix}$$

Fixed Angles

We can also represent an arbitrary orientation as a sequence of 3 elementary rotations w.r.t. to a **fixed frame of reference**. They are called **Fixed Angles**

Example:
Fixed Angles XYZ



Fixed Angles

We can also represent an arbitrary orientation as a sequence of 3 elementary rotations w.r.t. to a **fixed frame of reference**. They are called **Fixed Angles**

Example:

Fixed Angles XYZ

Rotation around

X

$$R_x(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\psi & -s\psi \\ 0 & s\psi & c\psi \end{bmatrix}$$

Rotation around

Y

$$R_y(\theta) = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}$$

Rotation around Z

$$R_z(\phi) = \begin{bmatrix} c\phi & -s\phi & 0 \\ s\phi & c\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_{XYZ_f} = R_x(\psi)R_y(\theta)R_z(\phi) = \begin{bmatrix} c\phi c\theta & -s\phi c\psi + c\phi s\theta s\psi & s\phi s\psi + c\phi s\theta c\psi \\ s\phi c\theta & c\phi c\psi + s\phi s\theta s\psi & -c\phi s\psi + s\phi s\theta c\psi \\ -s\theta & c\theta s\psi & c\theta c\psi \end{bmatrix}$$

With respect to Euler Angles the multiplication of the rotation matrices is done **right-to-left**. This means that the rotation matrices for Euler Angles ZYX and Fixed Angles XYZ have the same expression.

Orientation Representation Conversions

There are formulas that allow to go from one representation to another. For example:

Rotation Matrix

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

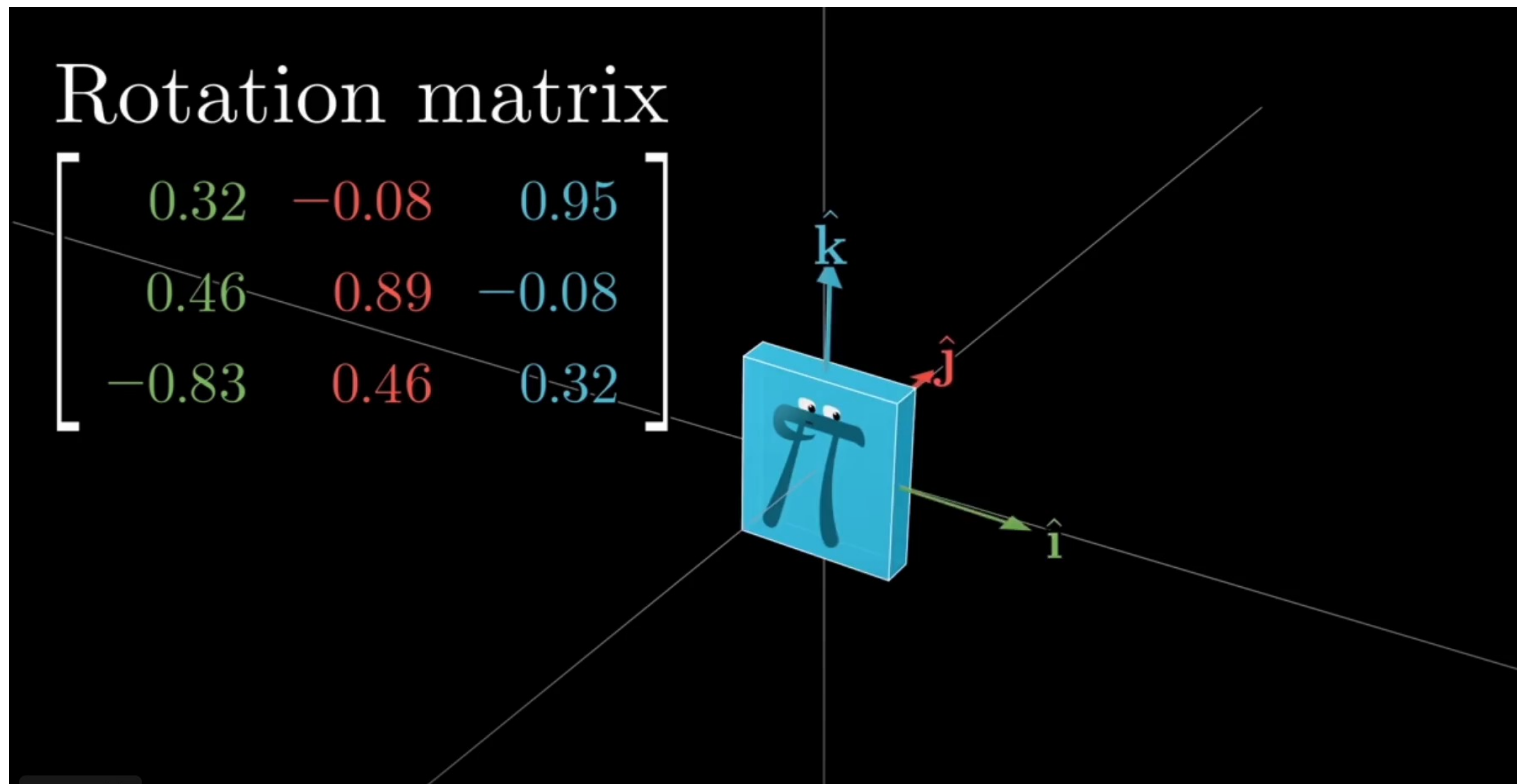


Fixed Angles XYZ

$$\begin{aligned} (\text{pitch}) \quad \theta &= \text{atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2}) \\ (\text{roll}) \quad \phi &= \text{atan2}\left(\frac{r_{21}}{\cos(\theta)}, \frac{r_{11}}{\cos(\theta)}\right) \\ (\text{yaw}) \quad \psi &= \text{atan2}\left(\frac{r_{32}}{\cos(\theta)}, \frac{r_{33}}{\cos(\theta)}\right) \end{aligned}$$

(Gimbal Lock) Singularity: When the pitch angle is $\theta = \pm 90^\circ$, the first (roll) and third (yaw) axes become aligned and can be collapsed to a single rotation. All representations with Euler Angles or Fixed Angles have this problem.

Gimbal Lock



Source: <https://www.youtube.com/watch?v=zjMulxRvygQ>

Unit Quaternion

Unit quaternions are a popular representation of orientation because they **do not have singularities**, unlike Euler and Fixed angles, albeit it is **not a minimal representation**

$$\mathbf{q} = (q_0, q_1, q_2, q_3) \quad \text{with} \quad \|\mathbf{q}\| = 1$$

Addition and multiplication operations for quaternions are defined as:

Addition

$$\mathbf{q} + \mathbf{v} = \begin{bmatrix} q_0 + v_0 \\ q_1 + v_1 \\ q_2 + v_2 \\ q_3 + v_3 \end{bmatrix}$$

Null element of addition

$$\mathbf{0} = (0, 0, 0, 0)$$

Multiplication

$$\mathbf{q}\mathbf{v} = \begin{bmatrix} q_0v_0 - q_1v_1 - q_2v_2 - q_3v_3 \\ q_0v_1 + q_1v_0 + q_2v_3 - q_3v_2 \\ q_0v_2 + q_2v_0 + q_3v_1 - q_1v_3 \\ q_0v_3 + q_3v_0 + q_1v_2 - q_2v_1 \end{bmatrix}$$

Identity element of multiplication

$$\mathbf{I} = (1, 0, 0, 0)$$

Unit Quaternion

The conjugate of a **unit quaternion** is $\tilde{q} = (q_0, -q_1, -q_2, -q_3)$, that is $q\tilde{q} = \tilde{q}q = 1$

The inverse of a **quaternion** is $q^{-1} = \frac{\tilde{q}}{\|q\|^2}$

For a unit
quaternion
this is 1.

Only if the
quaternion is
unitary

Conversions

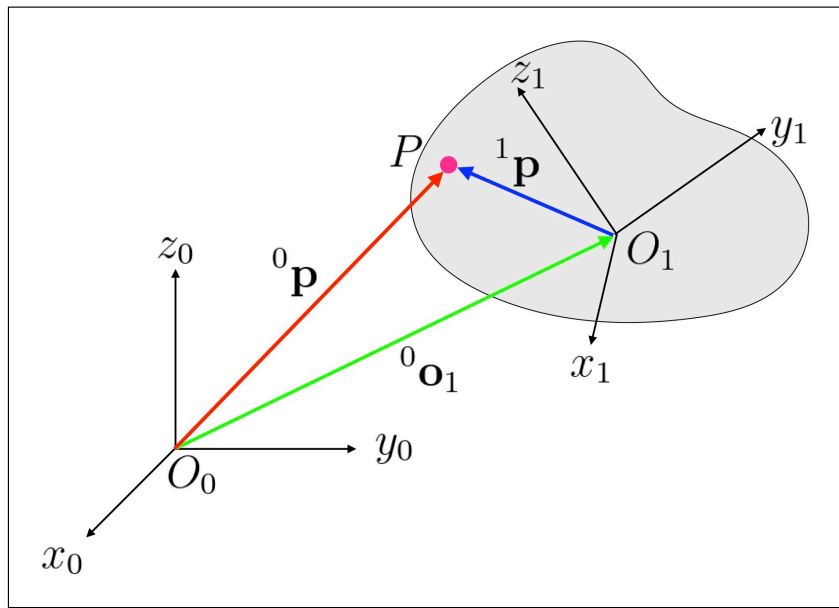
Quaternion -> Rotation Matrix

$$R = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}$$

Rotation Matrix -> Quaternion

$$\begin{aligned} q_0 &= \frac{1}{2} \sqrt{1 + r_{11} + r_{22} + r_{33}} \\ q_1 &= \frac{r_{32} - r_{23}}{4q_0} \\ q_2 &= \frac{r_{13} - r_{31}}{4q_0} \\ q_3 &= \frac{r_{21} - r_{12}}{4q_0} \end{aligned}$$

Homogeneous Representation



Combining translations and rotations, we can express the coordinates of a point P in frame F_0 from its coordinates in F_1

$${}^0\mathbf{p} = {}^0\mathbf{o}_1 + {}^0R_1 {}^1\mathbf{p}$$

The inverse transformation is

$${}^1\mathbf{p} = -{}^0R_1^T {}^0\mathbf{o}_1 + {}^0R_1^T {}^0\mathbf{p}$$

Homogeneous Representation

We can compactly write roto-translations using homogeneous representations.

$${}^0\mathbf{p} = {}^0\mathbf{o}_1 + {}^0R_1 {}^1\mathbf{p} \quad \longrightarrow \quad \begin{bmatrix} {}^0\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^0R_1 & {}^0\mathbf{o}_1 \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} {}^1\mathbf{p} \\ 1 \end{bmatrix}$$

Homogeneous representation
 ${}^0\tilde{\mathbf{p}}$
Homogeneous transformation
 0T_1

The **inverse** transformation is

$${}^1\tilde{\mathbf{p}} = \begin{bmatrix} {}^0R_1^T & -{}^0R_1^T {}^0\mathbf{o}_1 \\ \mathbf{0} & 1 \end{bmatrix} {}^0\tilde{\mathbf{p}} = {}^1T_0 {}^0\tilde{\mathbf{p}}$$

Time dependent rotations

Consider again two reference frames and suppose $\underline{F}_0$ is stationary whilst $\underline{F}_1$ rotates w.r.t. $\underline{F}_0$. Let us consider a point P attached to the rotating frame $\underline{F}_1$ and expressed with the constant vector ${}^1\mathbf{p}$

The coordinates of P in the stationary frame are:

$${}^0\mathbf{p}(t) = {}^0R_1(t){}^1\mathbf{p}$$

We can differentiate this with respect to time, but what is the derivative of a rotation matrix?

$${}^0\dot{\mathbf{p}}(t) = {}^0\dot{R}_1(t){}^1\mathbf{p}$$

Time dependent rotations

Since rotation matrices are orthogonal, we have that

$$R(t) R^T(t) = I \quad \Rightarrow \quad \dot{R}(t) R^T(t) + R(t) \dot{R}^T(t) = \mathbf{0}$$

We can define the new matrix

$$S(t) = \dot{R}(t) R^T(t)$$

Skew-symmetric matrix $S(\omega) = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & \omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$ with $\omega = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$

The **derivative of a rotation matrix** is $\dot{R}(t) = S(\omega(t))R(t)$

Time dependent rotations

From what we have seen, we can therefore write

$$\dot{\mathbf{p}} = \dot{R}(t)\mathbf{p}' = S(\omega(t))R(t)\mathbf{p}' = S(\omega(t))\mathbf{p}(t)$$

We also know that the velocity of the point P in the stationary frame is

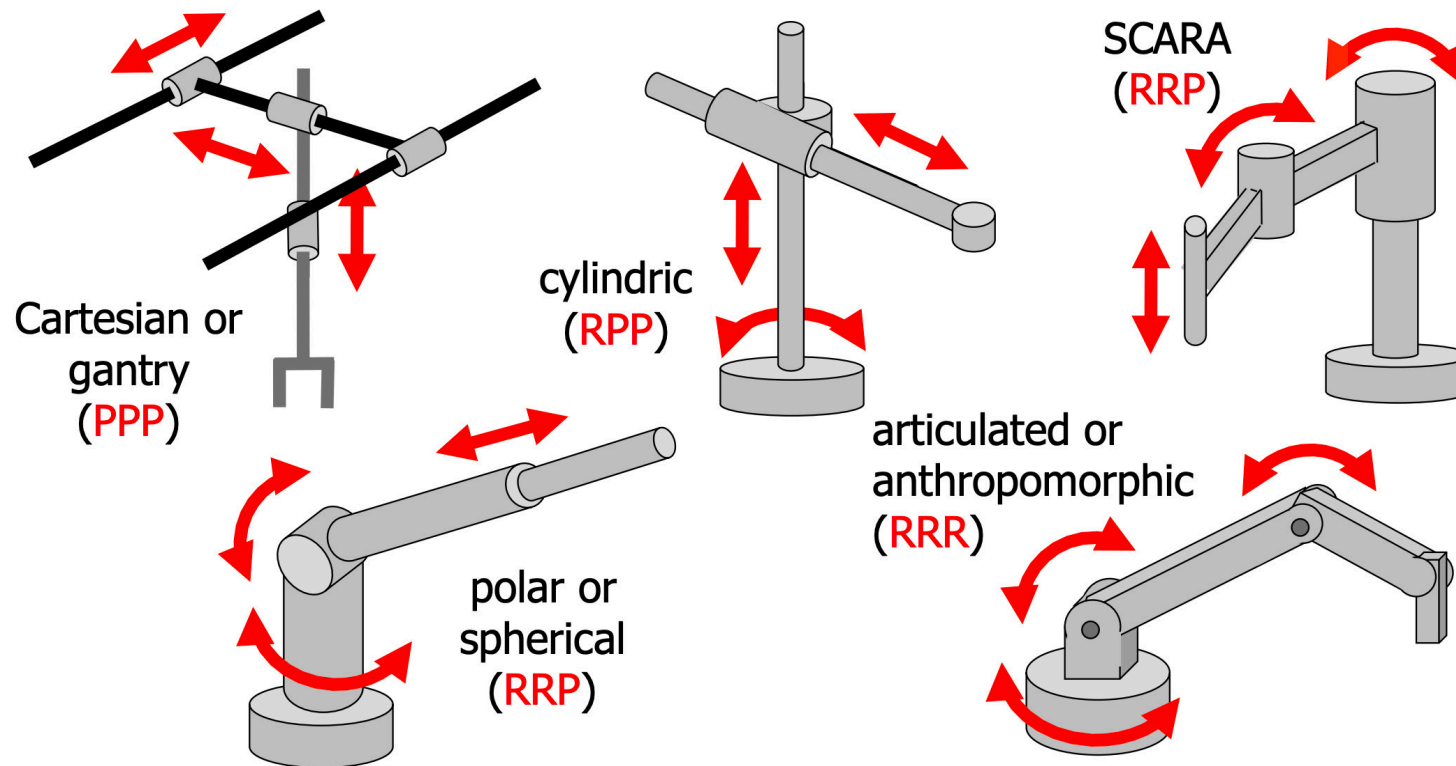
$$\dot{\mathbf{p}} = \omega(t) \times \mathbf{p}(t)$$

The skew-symmetric matrix can also be interpreted as the operator that computes the **cross product**

Kinematics of Serial Manipulators



Serial manipulators examples(first 3 DoFs)

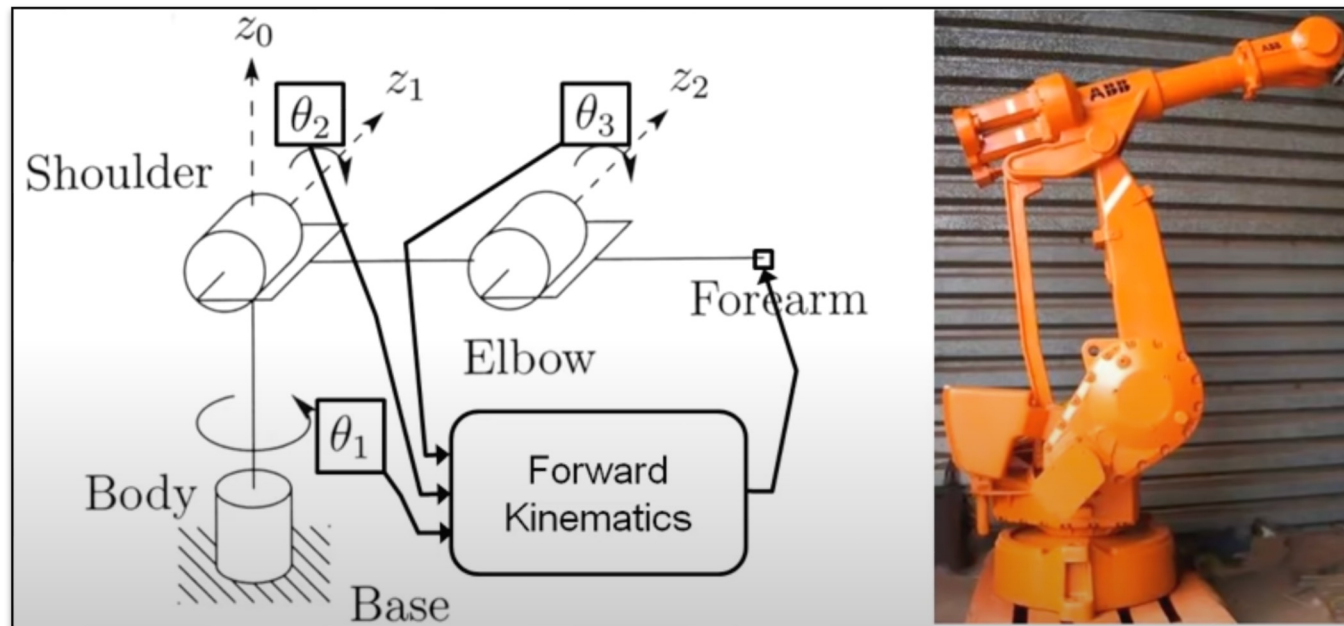


R = 1-dof rotational (**revolute**) joint
P = 1-dof translational (**prismatic**) joint

Forward Kinematics

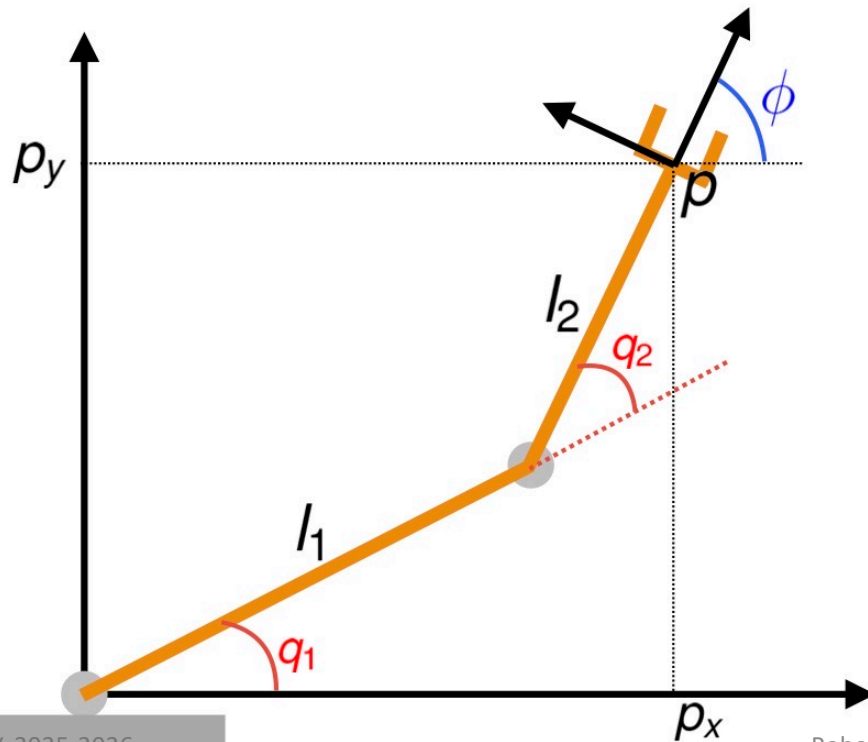
Forward Kinematics

- The **forward kinematics** $f(q)$ tells us the position and orientation of the tooltip point given a configuration of joint angles



Forward Kinematics: example by inspection

In some easy cases it is possible to derive the forward kinematics simply by inspection and geometric deduction



$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} p_x \\ p_y \\ \phi \end{bmatrix}$$

$$\begin{aligned} p_x &= l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) \\ p_y &= l_1 \sin(q_1) + l_2 \sin(q_1 + q_2) \\ \phi &= q_1 + q_2 \end{aligned}$$

Towards a systematic approach

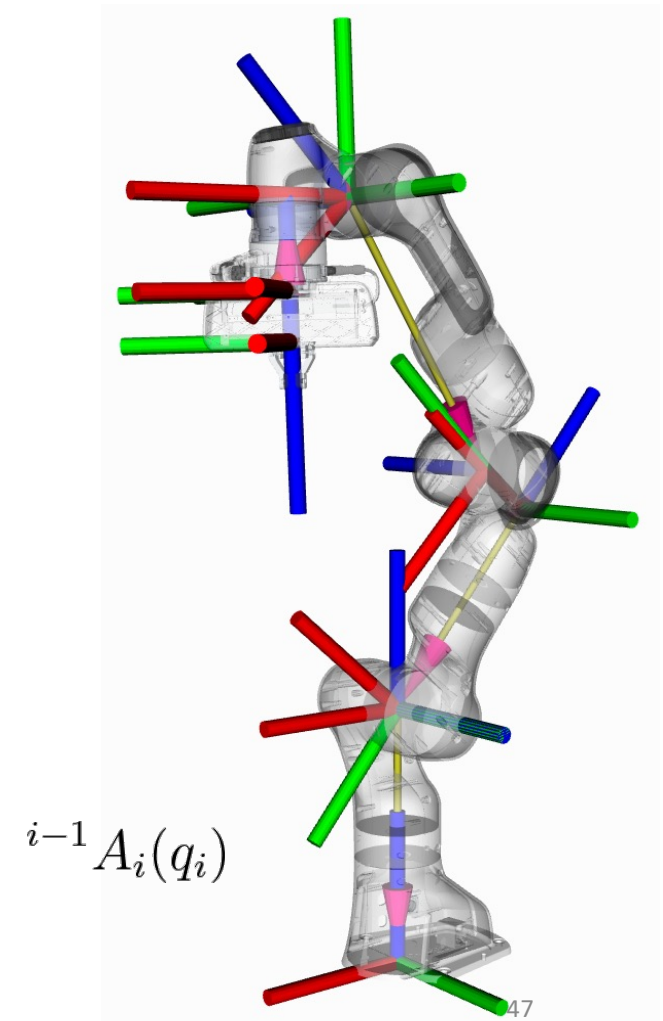
- Attach a frame of reference to each link.
- The transformation from a frame F_{i-1} to the next frame F_i is represented as a homogeneous that only depends on the joint variable q_i

$${}^{i-1}A_i(q_i)$$

- Proceeding iteratively

$${}^0T_n(\mathbf{q}) = {}^0A_1(q_1) \cdot {}^1A_2(q_2) \cdot \dots \cdot {}^{n-1}A_n(q_n)$$

How do we place the frames?



Denavit-Hartenberg convention

Denavit-Hartenberg established a convention to place the reference frames which collapses the 6 parameters to represent a homogeneous transformation matrix (3 position + 3 orientation) into only **4 link/joint parameters**.

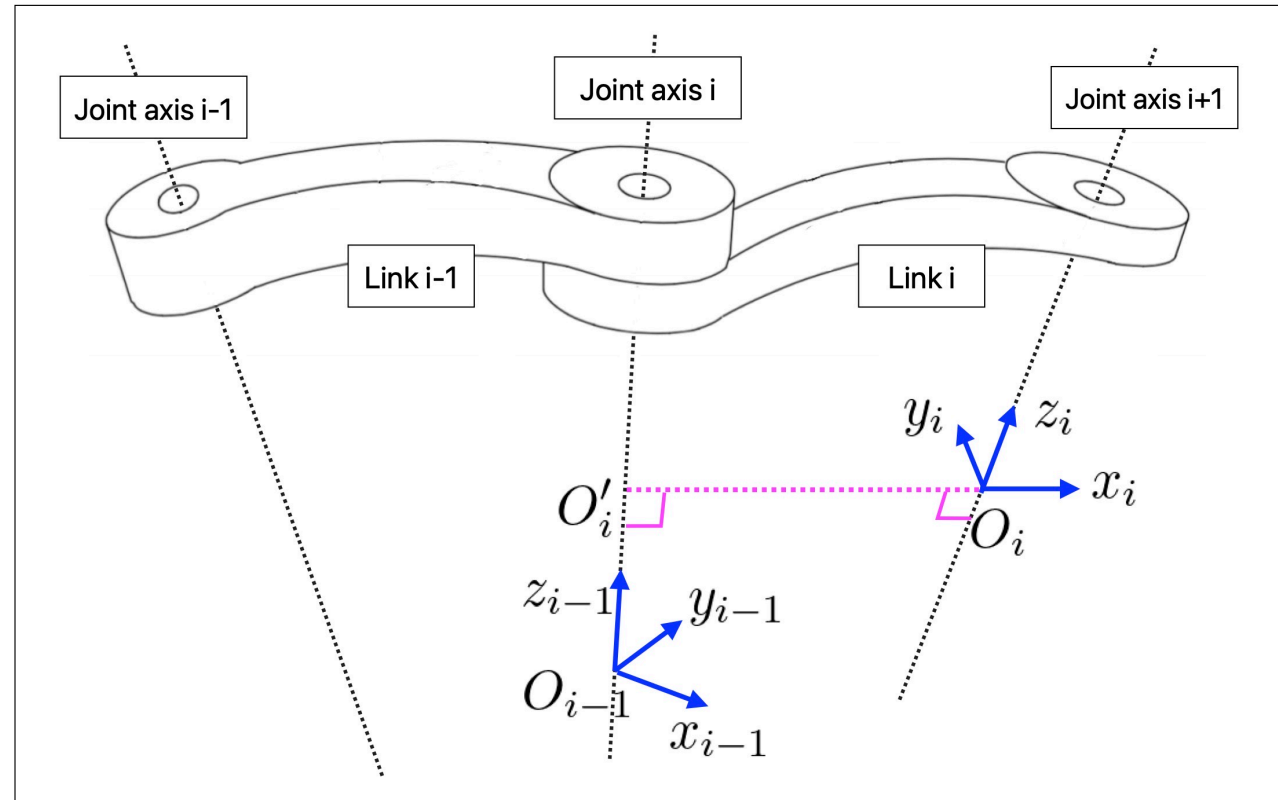
DH parameters

- *Link length* a_i
 - *Link twist* α_i
 - *Joint offset* d_i
 - *Joint angle* θ_i
- These parameters are **fixed** for a robot
- These parameters depend on the **joint variable**

Denavit-Hartenberg parameters

DH convention

- z_i lies along the axis of joint $i+1$
- O_i is at the intersection of axis $i+1$ with the **common normal** to axes i and $i+1$; we denote with O'_i the intersection of this common normal with axis i
- x_i is aligned with the common normal to axes i and $i+1$, with positive orientation from i to $i+1$
- y_i completes a right-handed frame

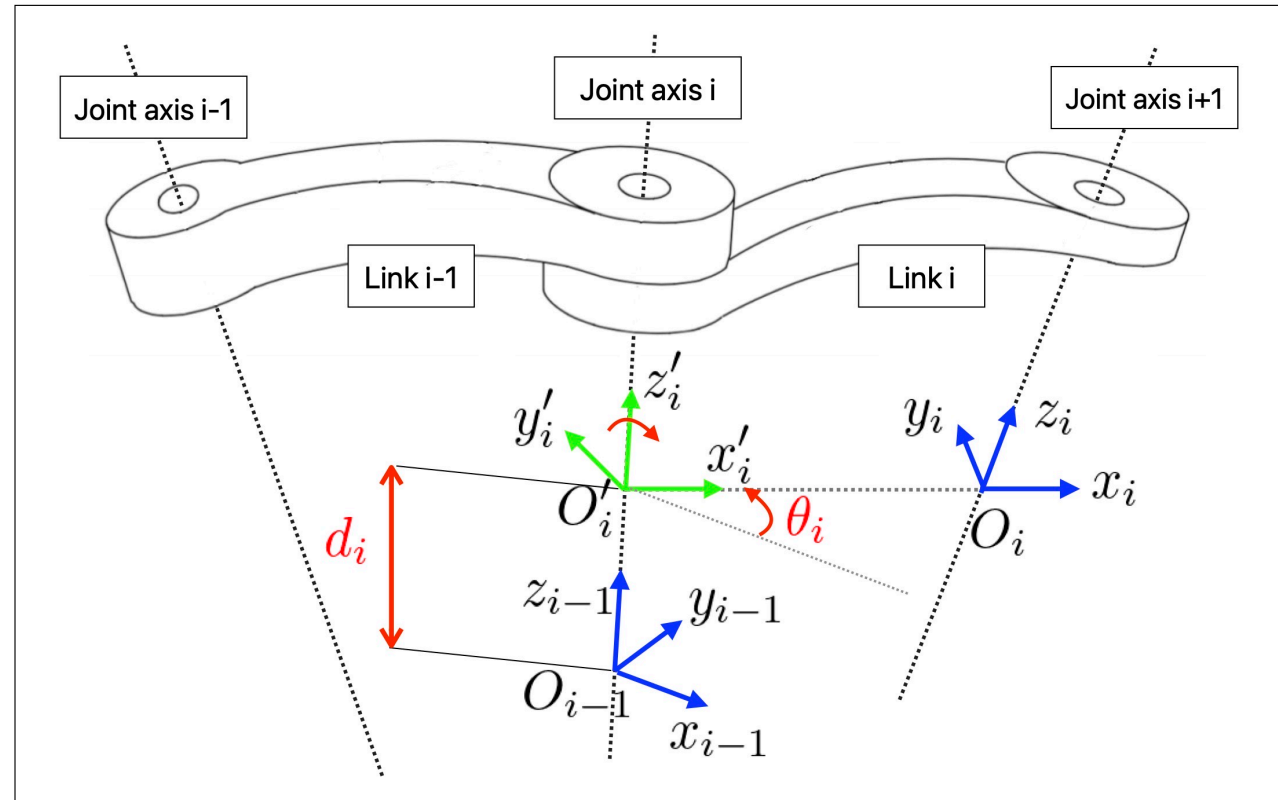


Denavit-Hartenberg parameters

The transformation from frame F_{i-1} to F_i can be obtained as a sequence of two roto-translations:

$$\mathcal{F}_{i-1} \Rightarrow \mathcal{F}'_i$$

- **Translate** F_{i-1} along z_{i-1} by d_i to superimpose O_{i-1} on O'_i
- **Rotate** the frame around z_{i-1} by θ_i to align x_{i-1} to x_i

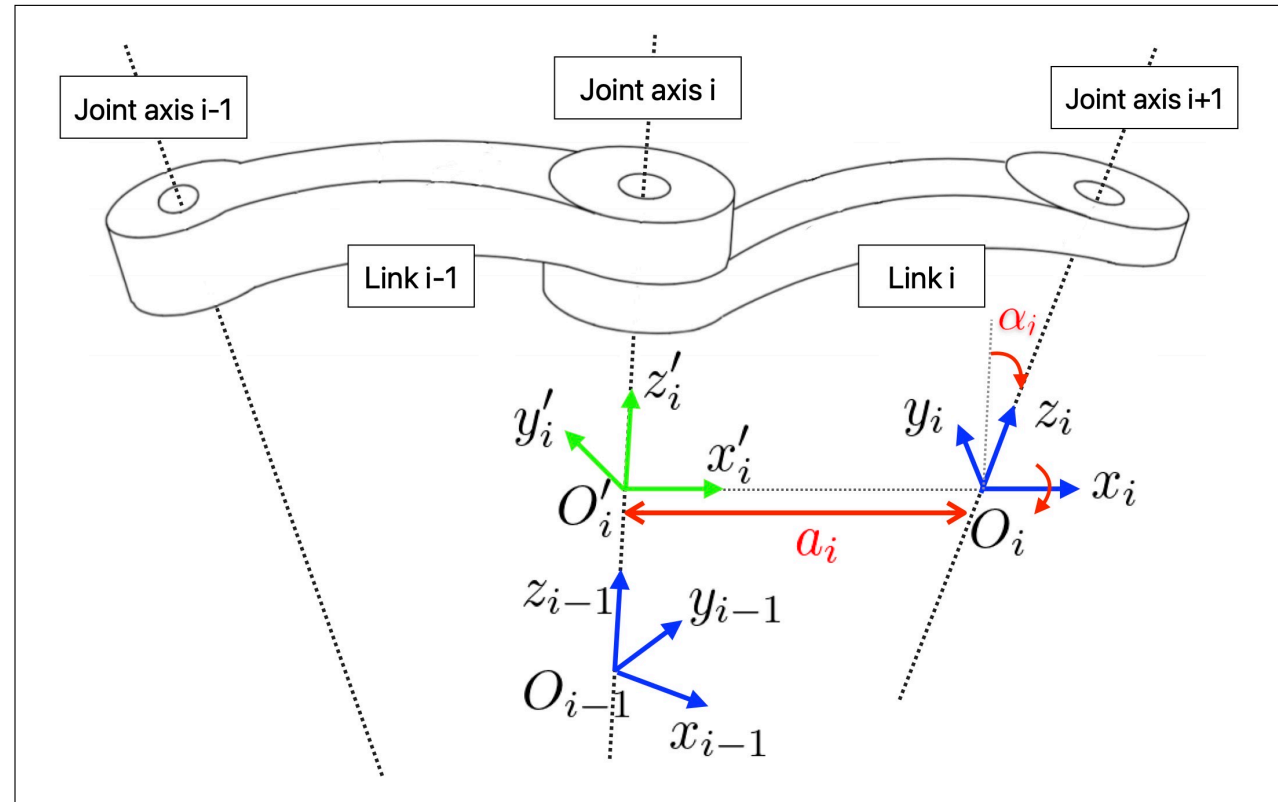


Denavit-Hartenberg parameters

The transformation from frame F_{i-1} to F_i can be obtained as a sequence of two roto-translations:

$$\mathcal{F}'_i \Rightarrow \mathcal{F}_i$$

- **Translate** F'_i along x'_i by a_i to superimpose O'_i on O_i
- **Rotate** the frame around x'_i by α_i to align z'_i to z_i



Denavit-Hartenberg parameters



Source: <https://www.youtube.com/watch?v=rA9tm0gTln8>

DH homogeneous transformation

We can express the two roto-translations using homogeneous transformations

1. $\mathcal{F}_{i-1} \Rightarrow \mathcal{F}'_i$

Translate along z_{i-1} by d_i and rotate
around z_{i-1} by θ_i

$${}^{i-1}A_{i'} = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & 0 \\ s\theta_i & c\theta_i & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. $\mathcal{F}'_i \Rightarrow \mathcal{F}_i$

Translate along x'_i by a_i and rotate
around x'_i by α_i

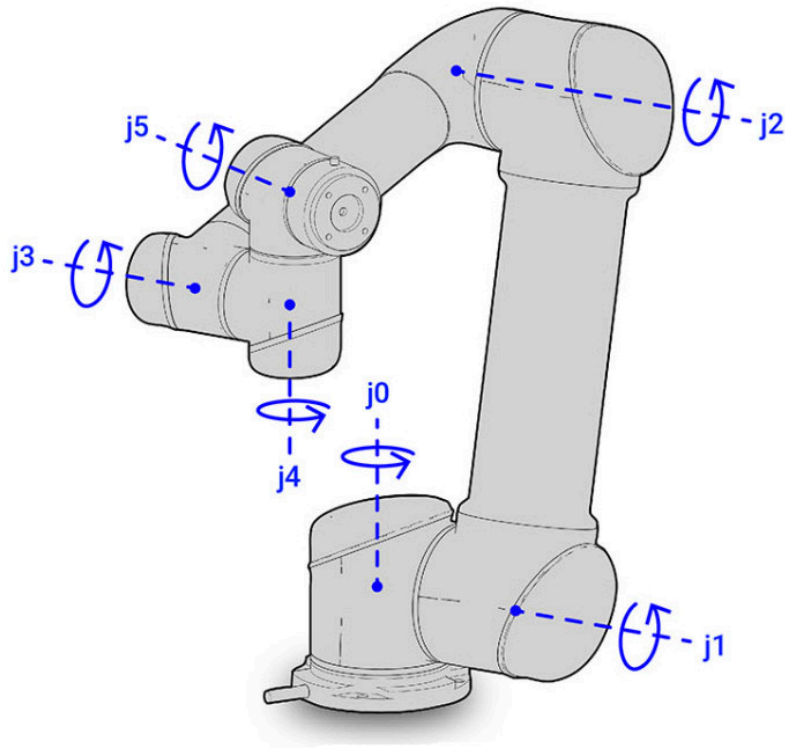
$${}^{i'}A_i = \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & c\alpha_i & -s\alpha_i & 0 \\ 0 & s\alpha_i & c\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

 ${}^{i-1}A_i(q_i) = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Homogeneous transformation of a joint using DH parameters. The joint variable q_i is:

- θ_i is for a rotational joint
- d_i for a prismatic joint

DH homogenous transformation



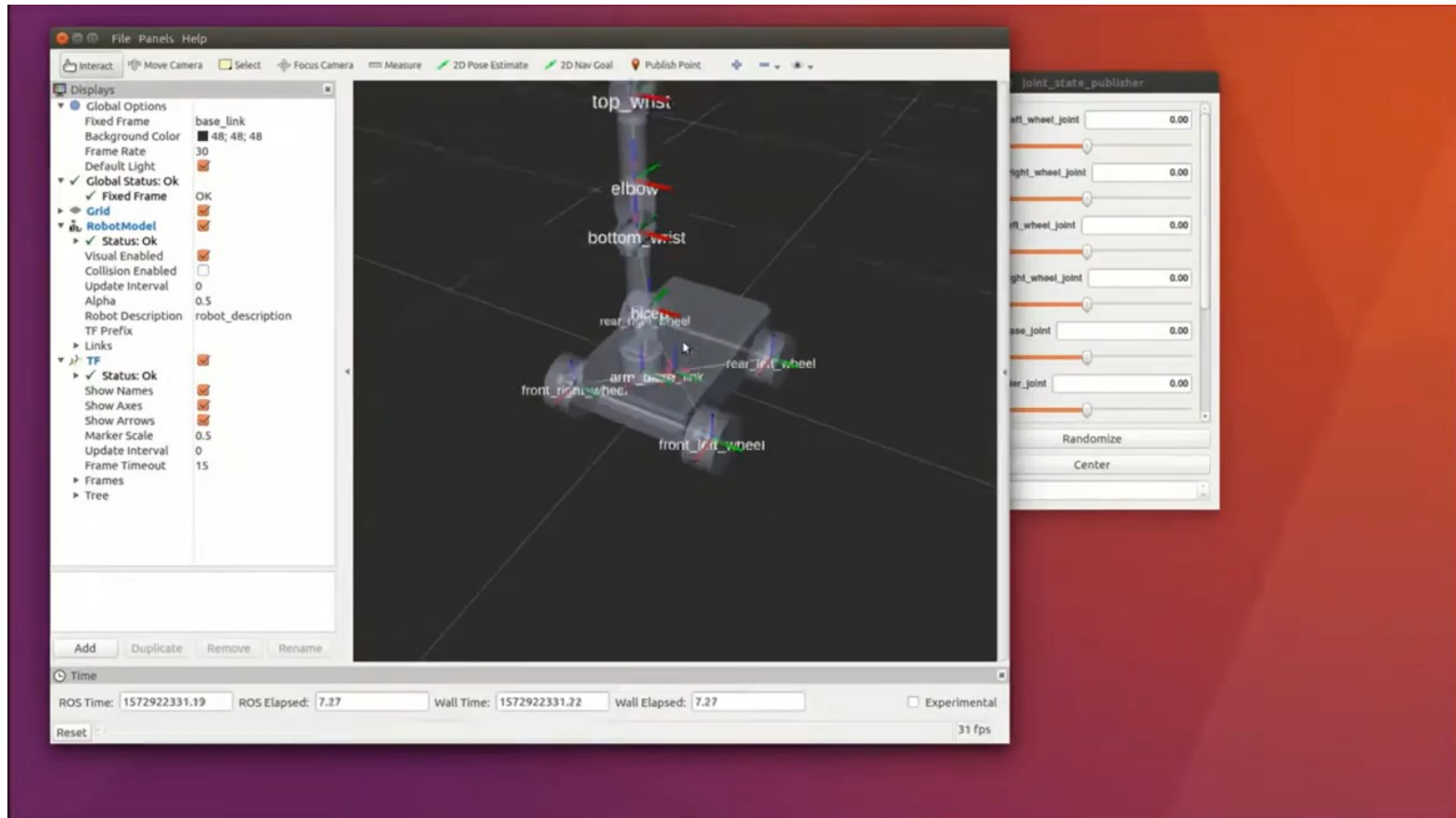
$${}^0T_n(\mathbf{q}) = {}^0A_1(q_1) \cdot {}^1A_2(q_2) \cdot \dots \cdot {}^{n-1}A_n(q_n)$$

$$= \begin{bmatrix} {}^0R_n & {}^0\mathbf{p} \\ \mathbf{0} & 1 \end{bmatrix}$$

This transformation provides the mapping from the joint variables q_i to the orientation 0R_n and position 0p of the end-effector w.r.t. the base frame F_0



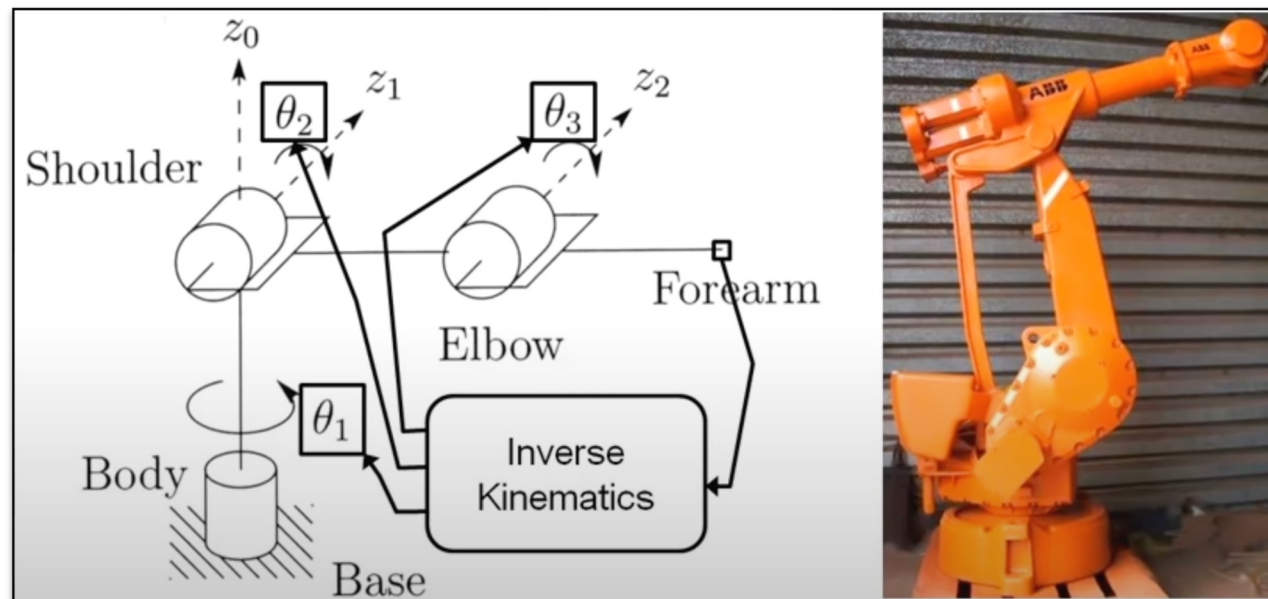
Forward Kinematics and ROS



Inverse Kinematics

Inverse Kinematics

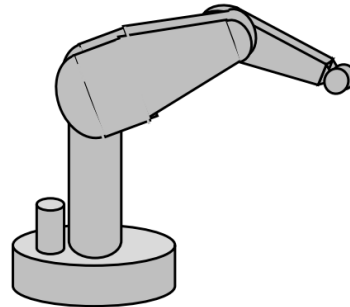
The **inverse kinematics** tells us the what are the joint angles (if any) that achieve a desired tooltip pose



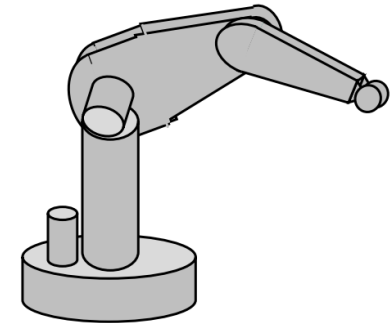
Inverse Kinematics: multiplicity of solutions

- the forward kinematics has one unique solution
- the inverse kinematics may have zero, multiple or even infinite solutions

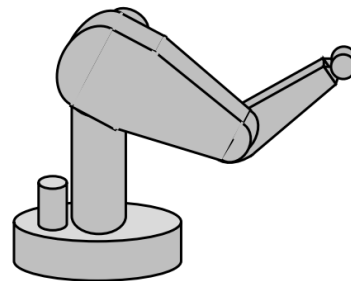
RIGHT UP



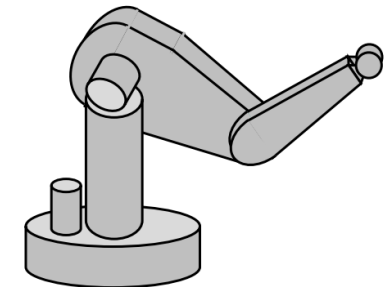
LEFT UP



RIGHT DOWN



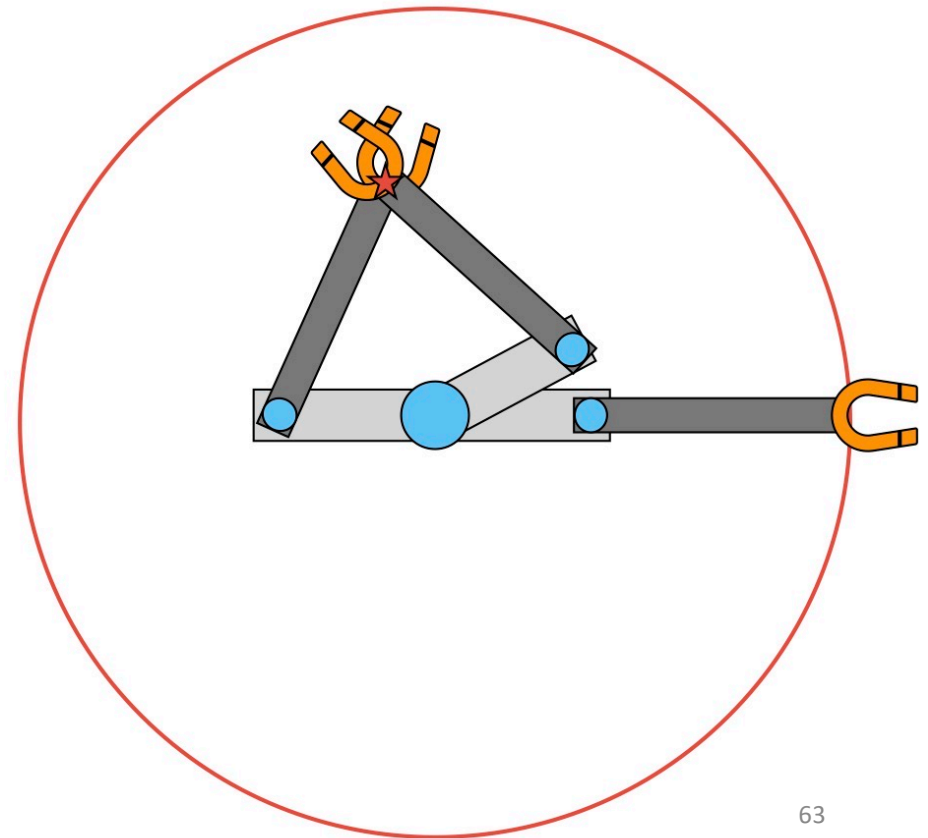
LEFT DOWN



Inverse Kinematics: possible cases

- $m = n$
 - There are no solutions
 - There is a **finite** number of solutions
 - Singular solutions (finite or infinite)
- $m < n$ (redundant case)
 - There are no solutions
 - ∞^{n-m} **solutions**
 - Singular solutions (finite or infinite)

Example: Non redundant case



Solving the Inverse Kinematics

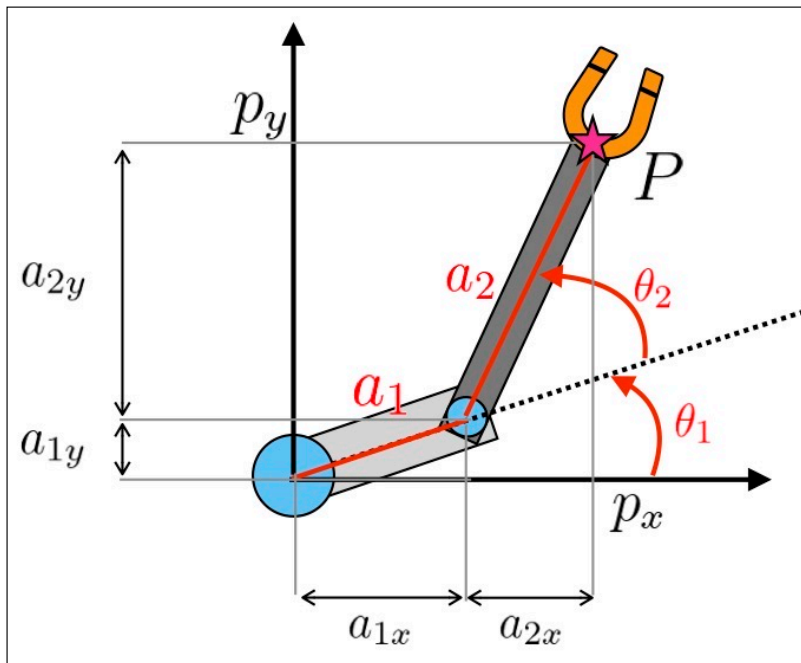
Analytical solution (closed form)

- Using **algebraic methods** or **geometric inspection**
- If applicable, it is preferred (faster, guarantees)

Numerical solution (iterative form)

- Using **Newton method**, **gradient method**, or other numerical methods to find the solution to $r_d = f(q)$
- Easier to setup and generalize to different configurations, but slower
- Necessary for redundant manipulators or close to singularities

Analytical Solution Example



$$p_x = a_{1x} + a_{2x} = a_1 \cos(\theta_1) + a_2 \cos(\theta_1 + \theta_2)$$

$$p_y = a_{1y} + a_{2y} = a_1 \sin(\theta_1) + a_2 \sin(\theta_1 + \theta_2)$$

Squaring
and
summing

$$c_2 = \frac{p_x^2 + p_y^2 - a_1^2 - a_2^2}{2 a_1 a_2}$$

$$\Rightarrow \theta_2 = \text{atan2}(s_2, c_2)$$

$$s_2 = \pm \sqrt{1 - c_2^2}$$

Two solutions (elbow
up/down)

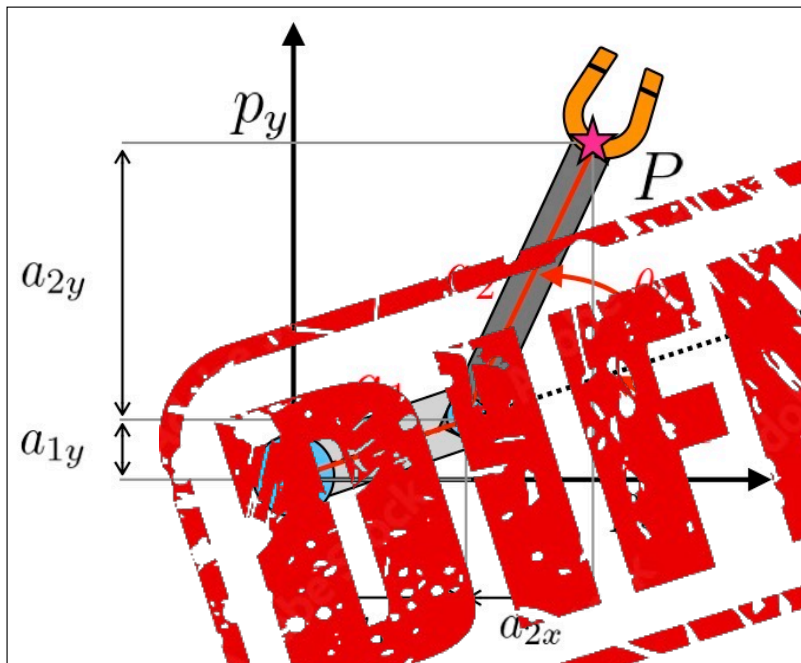
$$c_1 = \frac{(a_1 + a_2 c_2) p_x + a_2 s_2 p_y}{p_x^2 + p_y^2}$$

Finally

$$\Rightarrow \theta_1 = \text{atan2}(s_1, c_1)$$

$$s_1 = \frac{(a_1 + a_2 c_2) p_y - a_2 s_2 p_x}{p_x^2 + p_y^2}$$

Analytical Solution Example



$$p_x = a_{1x} + a_{2x} = a_1 \cos(\theta_1) + a_2 \cos(\theta_1 + \theta_2)$$

$$p_y = a_{1y} + a_{2y} = a_1 \sin(\theta_1) + a_2 \sin(\theta_1 + \theta_2)$$

Solving
and
squaring

$$\Rightarrow \theta_2 = \text{atan2}(s_2, c_2)$$

Two solutions (elbow
up/down)

$$c_1 = \frac{(a_1 + a_2 c_2)p_x + a_2 s_2 p_y}{p_x^2 + p_y^2}$$

Finally

$$\Rightarrow \theta_1 = \text{atan2}(s_1, c_1)$$

$$s_1 = \frac{(a_1 + a_2 c_2)p_y - a_2 s_2 p_x}{p_x^2 + p_y^2}$$

Numerical Solution: Newton method

- We want to find a solution to $x_d = f(q)$
- Let's define $J_A(q) = \frac{\partial f(q)}{\partial q}$ **Analytical Jacobian**

- **Newton Method:**

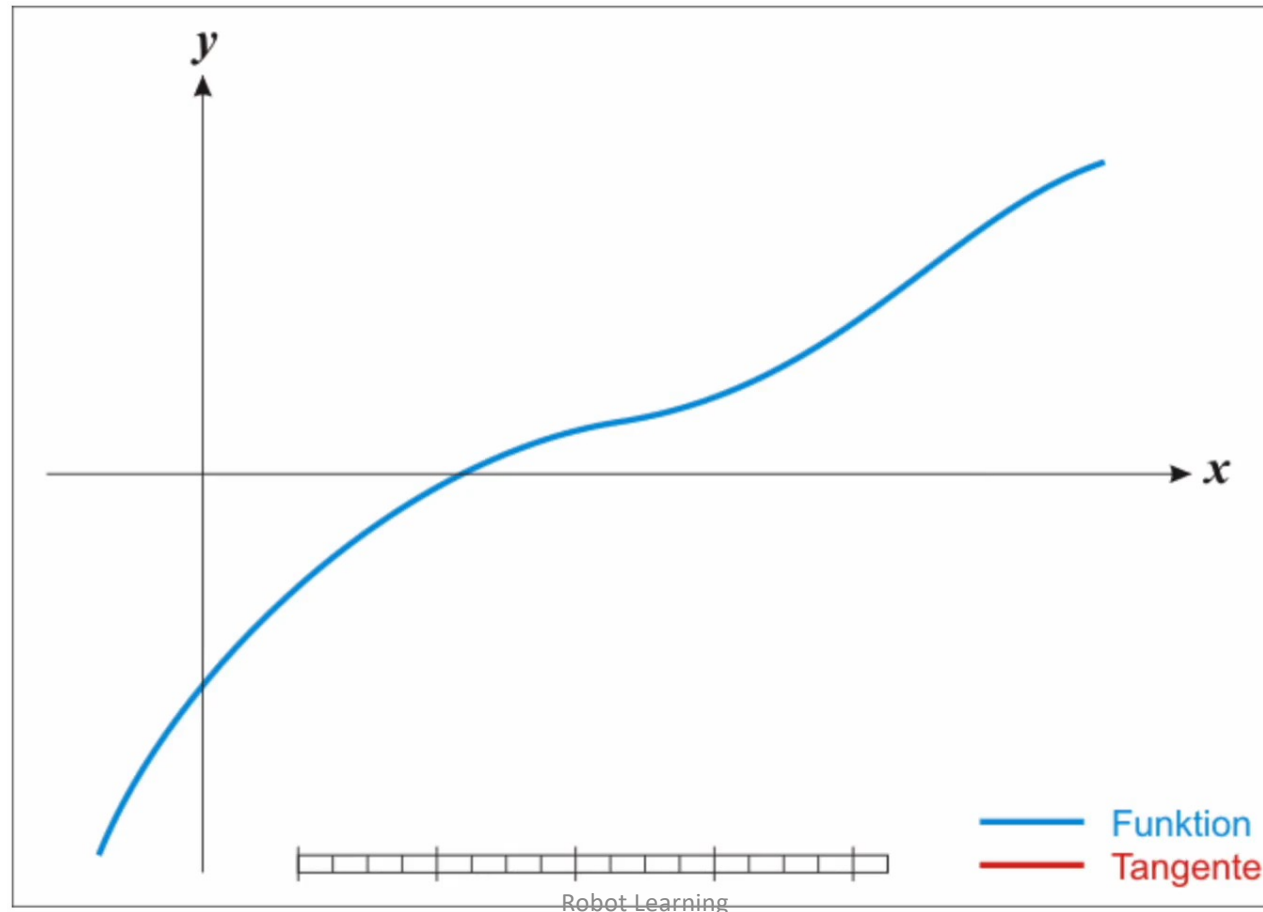
$$x_d = f(q^k) + J_A(q)(q - q^k) + O(\|q - q^k\|^2)$$



$$q^{k+1} = f(q^k) + J_A^{-1}(q^k)[x_d - f(q^k)]$$

- If the robot is redundant ($m < n$), instead of the inverse we should use the **pseudoinverse** $J_A^\dagger = (J_A^T J_A)^{-1} J_A^T$

Numerical Solution: Newton method



Numerical Solution: Gradient method

- Consider the objective function

$$H(\mathbf{q}) = \frac{1}{2} \|x_d - f(\mathbf{q})\|^2 = \frac{1}{2} (x_d - f(\mathbf{q}))^T (x_d - f(\mathbf{q}))$$

- and minimize it using a gradient descent

$$\mathbf{q}^{k+1} = \mathbf{q}^k + \alpha \nabla_{\mathbf{q}} H(\mathbf{q}^k) \quad \text{with} \quad \nabla_{\mathbf{q}} H(\mathbf{q}) = -J_A^T(\mathbf{q}) [x_d - f(\mathbf{q})]$$



- Gradient Method:** $\mathbf{q}^{k+1} = \mathbf{q}^k - \alpha J_A^T(\mathbf{q}) [x_d - f(\mathbf{q}^k)]$
- Note:** if the step size $\alpha > 0$ is too big, we may miss the minimum

Newton vs. Gradient method

- Gradient method

- **Computationally simpler:** it requires a matrix transpose instead of the (pseudo-)inverse
- It never diverges but it only has **linear convergence**

- Newton method

- **Computationally expensive**
- no convergence guarantee, but if the initial guess is close to the solution, it has a **quadratic convergence**

- An effective implementation could be to use initially the **Gradient method** to get close to the solution, and then switch to **Newton method**

Differential Kinematics

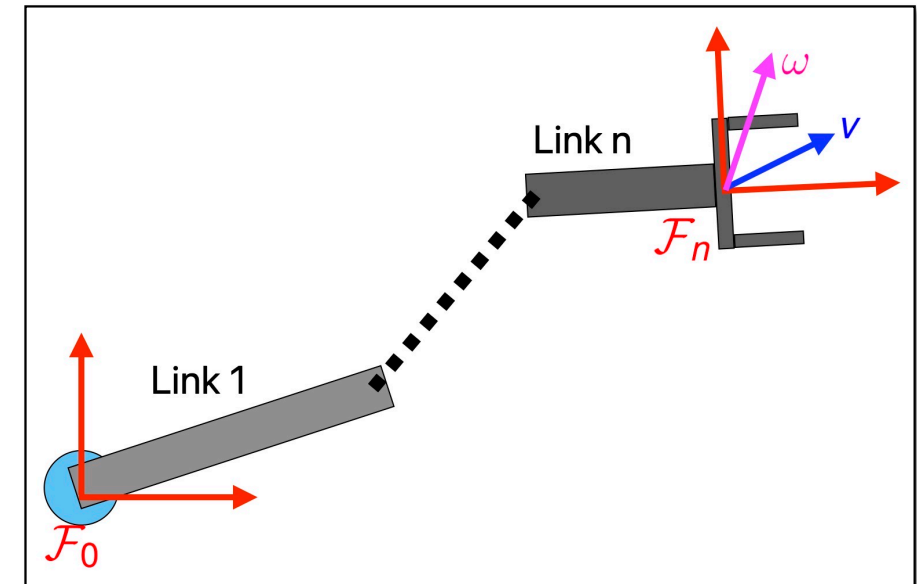
Differential Kinematics

The goal of **differential kinematics** is to express the linear velocity v and angular velocity ω of the end-effector in terms of the joint velocities and joint angles

We seek a mapping in the form

$$\begin{aligned} v &= J_P(q)\dot{q} \\ \omega &= J_O(q)\dot{q} \end{aligned} \Rightarrow \begin{bmatrix} v \\ \omega \end{bmatrix} = \underbrace{\begin{bmatrix} J_P(q) \\ J_O(q) \end{bmatrix}}_{J(q)} \dot{q}$$

Generalized coordinates



J is called **Geometric Jacobian**, to distinguish it from the Analytical Jacobian (seen earlier)

Geometrical Jacobian

To derive the **geometric Jacobian**, let's partition it into n columns (i.e., one for each joint)

$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} J_{P1}(\mathbf{q}) & \dots & J_{Pn}(\mathbf{q}) \\ J_{O1}(\mathbf{q}) & \dots & J_{On}(\mathbf{q}) \end{bmatrix} \dot{\mathbf{q}}$$

We have

Contribution of the i -th joint to the linear velocity



$$\mathbf{v} = J_{P1}(\mathbf{q})\dot{q}_1 + \dots + J_{Pi}(\mathbf{q})\dot{q}_i + \dots + J_{Pn}(\mathbf{q})\dot{q}_n$$

$$\boldsymbol{\omega} = J_{O1}(\mathbf{q})\dot{q}_1 + \dots + J_{Oi}(\mathbf{q})\dot{q}_i + \dots + J_{On}(\mathbf{q})\dot{q}_n$$



Contribution of the i -th joint to the angular velocity

There is a **superposition of effects** from the individual joints

Geometrical Jacobian

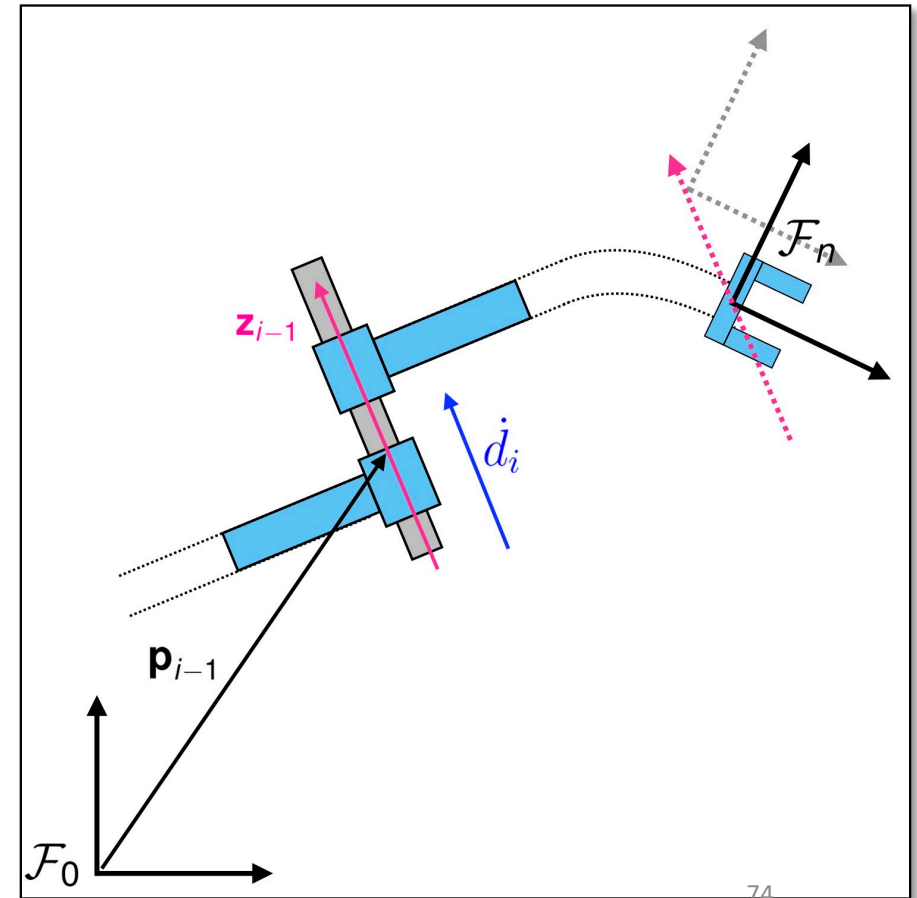
Prismatic Joints

- **Angular velocity:** a prismatic joint gives no contribution

$$J_{Oi}(\mathbf{q}) \dot{q}_i = \mathbf{0} \quad \Rightarrow \quad J_{Oi}(\mathbf{q}) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

- **Linear velocity:** the joint's contribution is

$$J_{Pi}(\mathbf{q}) \dot{q}_i = \dot{d}_i \mathbf{z}_{i-1} \quad \Rightarrow \quad J_{Pi}(\mathbf{q}) = \mathbf{z}_{i-1}$$



Geometrical Jacobian

Rotational Joints

- **Angular velocity:** the joint contribution is

$$J_{O_i}(\mathbf{q}) \dot{q}_i = \dot{\theta}_i \mathbf{z}_{i-1} \Rightarrow J_{O_i}(\mathbf{q}) = \mathbf{z}_{i-1}$$

- **Linear velocity:** the joint's contribution is

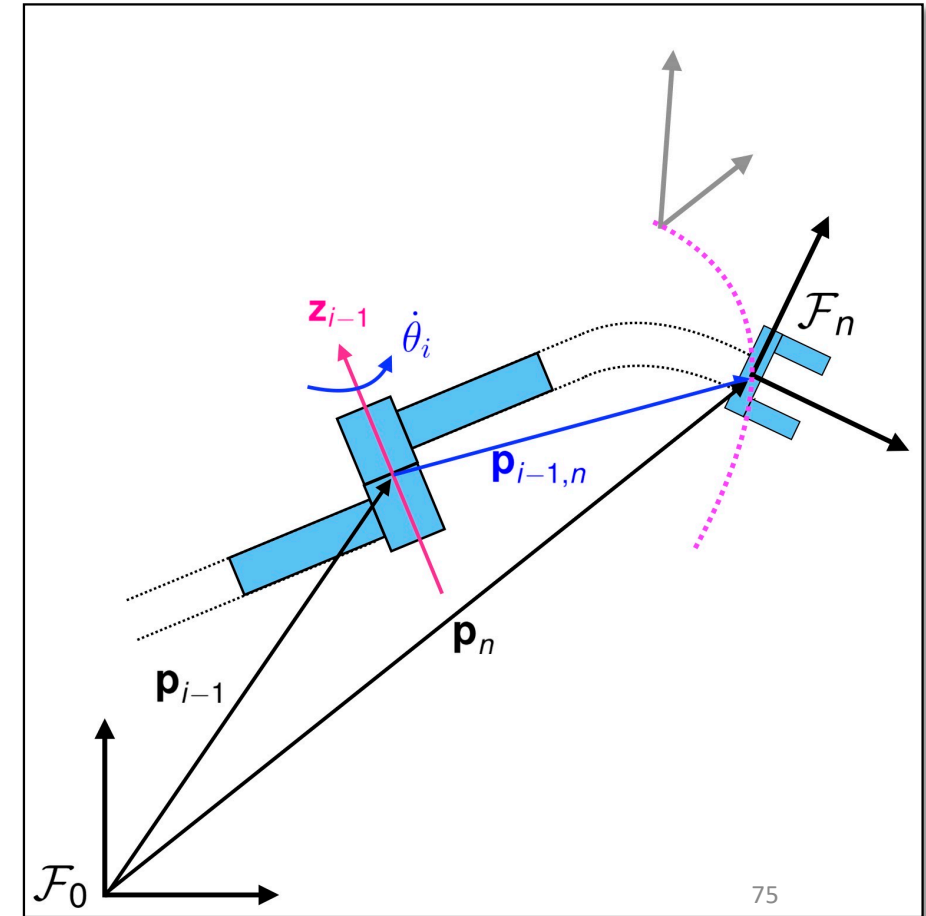
$$J_{P_i}(\mathbf{q}) \dot{q}_i = \dot{\theta}_i \mathbf{z}_{i-1} \times (\mathbf{p}_n - \mathbf{p}_{i-1})$$

Endpoint pose



Pose link i-1

$$J_{P_i}(\mathbf{q}) = \mathbf{z}_{i-1} \times (\mathbf{p}_n - \mathbf{p}_{i-1})$$



Geometric Jacobian

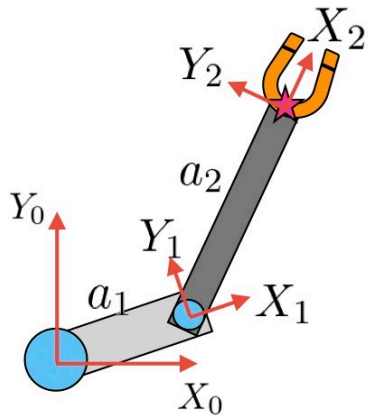
We can thus compute the Geometric Jacobian **one column at a time**:

$$\begin{bmatrix} J_P(\mathbf{q}) \\ J_O(\mathbf{q}) \end{bmatrix} = \begin{cases} \begin{bmatrix} \mathbf{z}_{i-1} \\ \mathbf{0} \end{bmatrix} & \text{Prismatic joint} \\ \begin{bmatrix} \mathbf{z}_{i-1} \times (\mathbf{p}_n - \mathbf{p}_{i-1}) \\ \mathbf{z}_{i-1} \end{bmatrix} & \text{Rotational joint} \end{cases}$$

All the vectors in these terms can be computed from the forward kinematics

$$\begin{aligned} \mathbf{z}_{i-1} &= {}^0R_1(q_1) \cdot \dots \cdot {}^{i-2}R_{i-1}(q_{i-1}) \cdot \mathbf{z}_0 \\ \tilde{\mathbf{p}}_n &= {}^0A_1(q_1) \cdot \dots \cdot {}^{n-1}A_n(q_n) \cdot \tilde{\mathbf{p}}_0 \\ \tilde{\mathbf{p}}_{i-1} &= {}^0A_1(q_1) \cdot \dots \cdot {}^{i-2}A_{i-1}(q_{i-1}) \cdot \tilde{\mathbf{p}}_0 \end{aligned} \quad \text{where} \quad \mathbf{z}_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{p}}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{p}}_n = \begin{bmatrix} \mathbf{p}_n \\ 1 \end{bmatrix}$$

Geometric Jacobian – Example planar 2R arm



DH parameters

joint	α_i	d_i	a_i	θ_i
1	0	0	a_1	q_1
2	0	0	a_2	q_2

Geometric Jacobian

$$J = \begin{bmatrix} \mathbf{z}_0 \times (\mathbf{p}_n - \mathbf{p}_0) & \mathbf{z}_1 \times (\mathbf{p}_n - \mathbf{p}_1) \\ \mathbf{z}_0 & \mathbf{z}_1 \end{bmatrix}$$

$$\mathbf{z}_0 = \mathbf{z}_1 = \mathbf{z}_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$${}^0A_1 = \begin{bmatrix} \cos(q_1) & -\sin(q_1) & 0 & a_1 \cos(q_1) \\ \sin(q_1) & \cos(q_1) & 0 & a_1 \sin(q_1) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{p}_1$$

$$\mathbf{p}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$${}^0A_2 = \begin{bmatrix} \cos(q_1 + q_2) & -\sin(q_1 + q_2) & 0 & a_1 \cos(q_1) + a_2 \cos(q_1 + q_2) \\ \sin(q_1 + q_2) & \cos(q_1 + q_2) & 0 & a_1 \sin(q_1) + a_2 \sin(q_1 + q_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{p}_2 = \mathbf{p}_n$$

Analytical vs. Geometric Jacobian

Geometric Jacobian

$$\begin{bmatrix} \dot{\mathbf{p}} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} J_P(\mathbf{q}) \\ J_O(\mathbf{q}) \end{bmatrix} \dot{\mathbf{q}} = J(\mathbf{q}) \dot{\mathbf{q}}$$

Analytic Jacobian

$$\begin{bmatrix} \dot{\mathbf{p}} \\ \dot{\boldsymbol{\phi}} \end{bmatrix} = \begin{bmatrix} J_P(\mathbf{q}) \\ J_\phi(\mathbf{q}) \end{bmatrix} \dot{\mathbf{q}} = J_A(\mathbf{q}) \dot{\mathbf{q}}$$

There is a mapping from the **derivative of the angles $\boldsymbol{\phi}$** and the **angular velocity $\boldsymbol{\omega}$**

$$\boldsymbol{\omega} = T(\mathbf{q})\dot{\mathbf{q}} \quad \Rightarrow \quad J(\mathbf{q}) = \begin{bmatrix} I & \mathbf{0} \\ \mathbf{0} & T(\mathbf{q}) \end{bmatrix} J_A(\mathbf{q})$$

- The expression of matrix ***T*** **depends on the representation used for $\boldsymbol{\phi}$** . For example, for **ZYZ Euler angles** it is:

$$T(\boldsymbol{\phi}) = \begin{bmatrix} 0 & -s\phi & c\phi s\theta \\ 0 & c\phi & s\phi s\theta \\ 1 & 0 & c\theta \end{bmatrix}$$

Kinematic Singularities

- Given the differential kinematic mapping

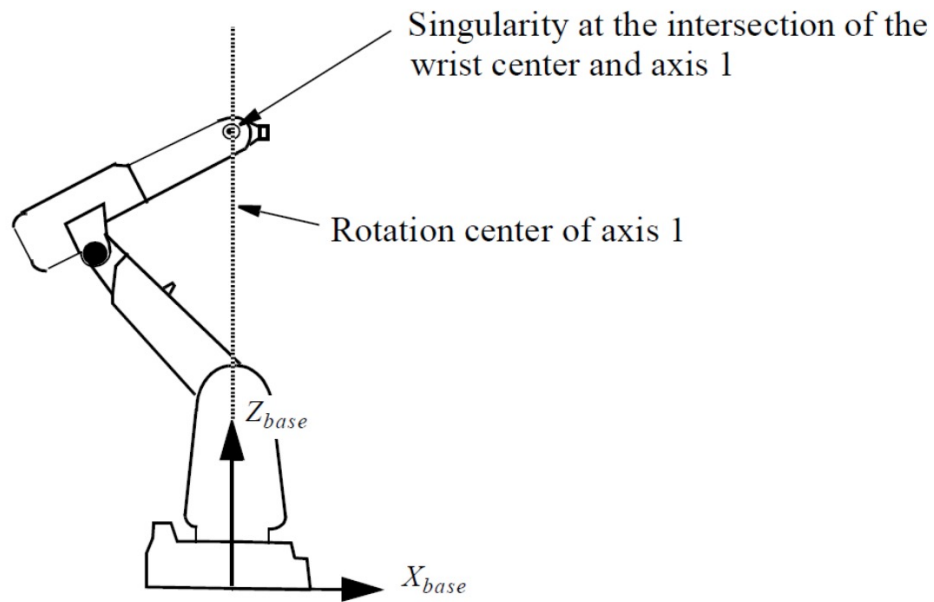
$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J(\mathbf{q})\dot{\mathbf{q}}$$

the values $\mathbf{q}$ for which the matrix J is rank-deficient are called **kinematic singularities**.

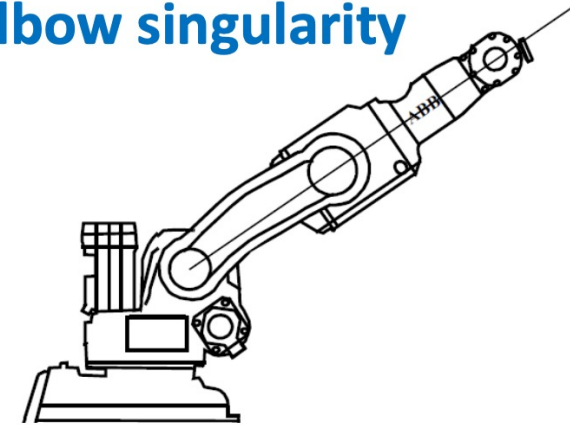
- At a singularity, we may have:
 - Loss of mobility** (some instantaneous movements are infeasible)
 - Possible **infinite solutions** of the inverse kinematics
 - High-velocities** in joint space
- Singularities may happen both at the **border** of the workspace, or **inside** the workspace

Kinematic Singularities - Examples

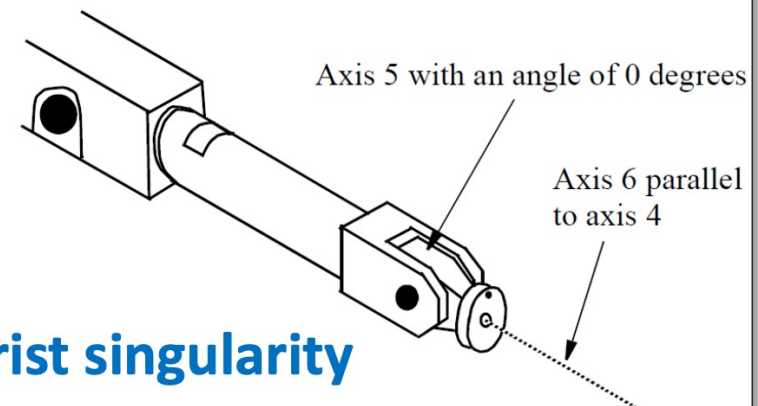
Shoulder singularity



Elbow singularity



Wrist singularity



Kinematic Singularities - Examples



Source: <https://youtu.be/ID2HQcxNoA?si=6ayDcdU8CINOmsjt>

Inversion of differential kinematics

- We can invert the differential kinematics to *find the joint velocities corresponding at a task space velocity for a specific robot configuration*

$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = J(\mathbf{q})\dot{\mathbf{q}} \quad \Rightarrow \quad \dot{\mathbf{q}} = J(\mathbf{q})^{-1} \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix}$$

Or the pseudo-inverse instead of the inverse

- Using SVD decomposition: $J = USV^T \Rightarrow J^{-1} = VS^{-1}U^T$

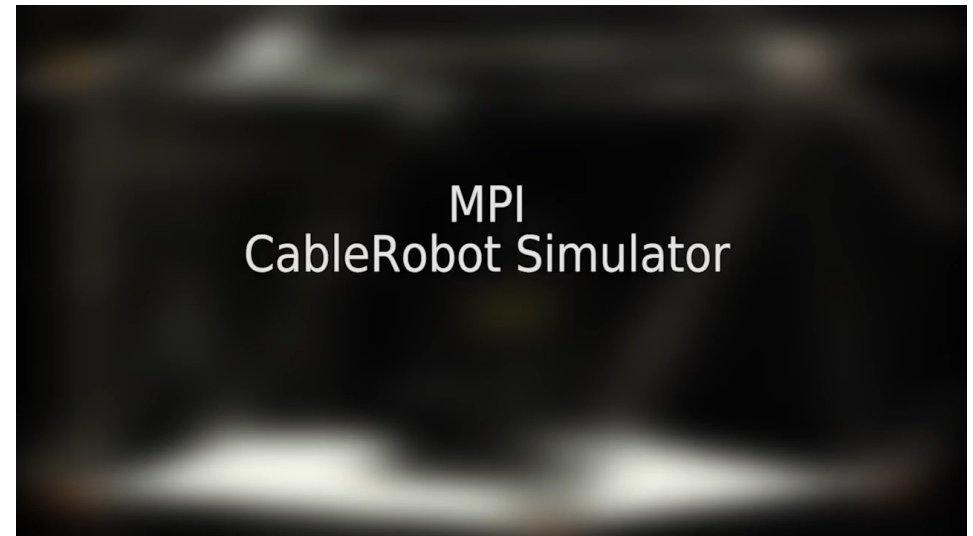
$$S = \begin{bmatrix} s_1 & & \\ & \ddots & \\ & & s_n \end{bmatrix} \xrightarrow{\text{blue arrow}} S^{-1} = \begin{bmatrix} \frac{1}{s_1} & & \\ & \ddots & \\ & & \frac{1}{s_n} \end{bmatrix}$$

Close to a **singularity** (s_1 close to 0), the term $1/s_1$ may explode, yielding very large joint velocities!

Parallel Manipulators

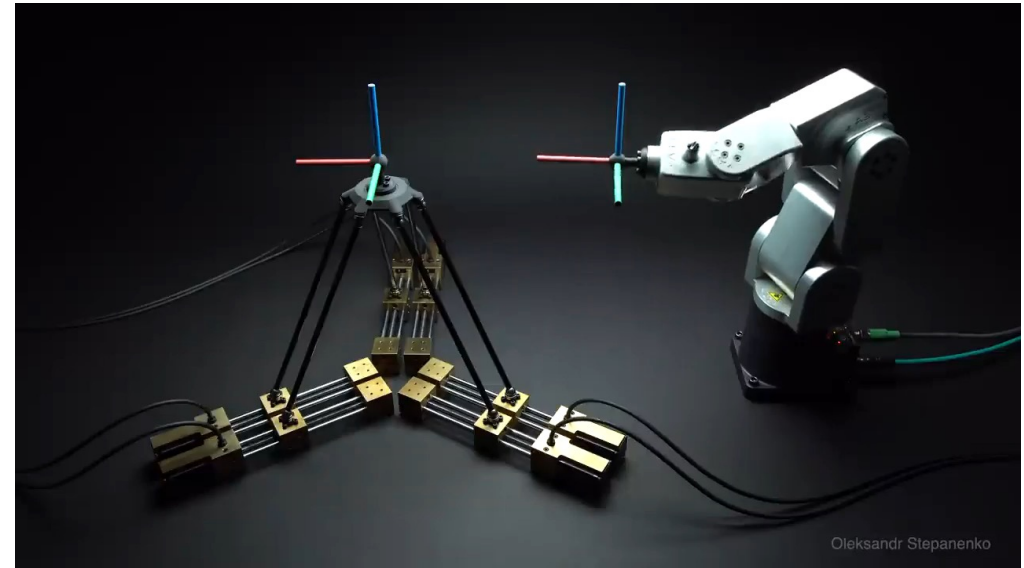
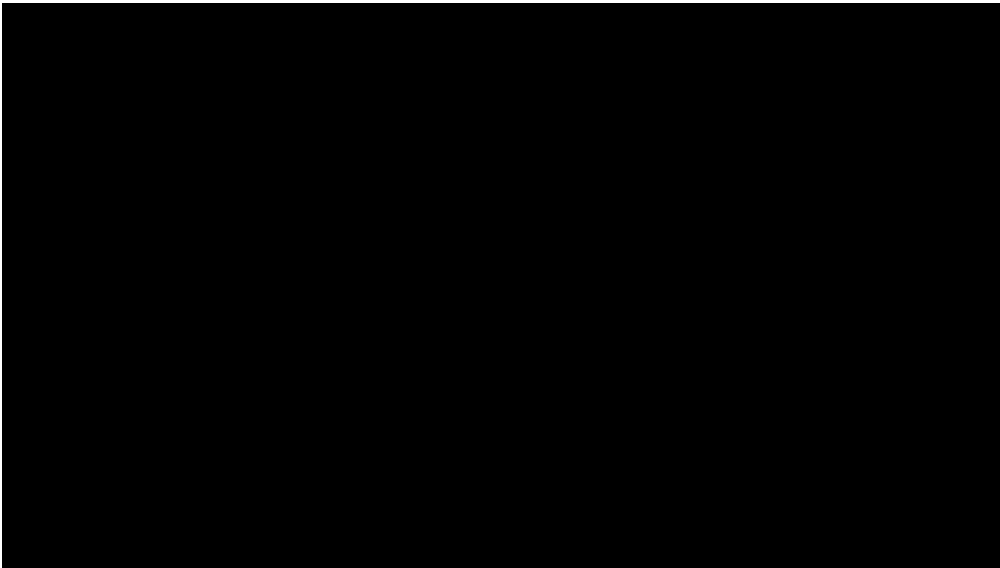


Parallel manipulators



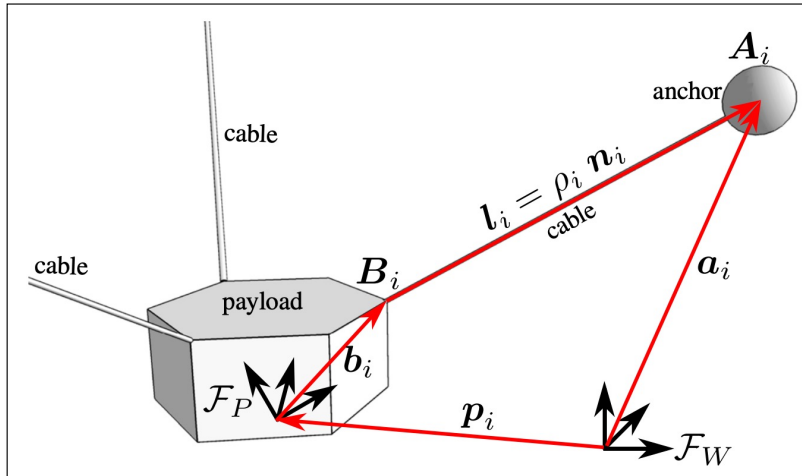
How a parallel robot moves?

Example: *hexapod platforms*



Inverse kinematics

The solution of the inverse kinematic problem is usually straightforward for parallel robots. For each leg/cable we can write a **loop-closure constraint**



Loop-closure constraint

$$l_i = \rho_i \mathbf{n}_i = \mathbf{a}_i - \mathbf{p} - {}^W R_P \mathbf{b}_i$$

- $\mathbf{b}_i$ **Fixed** position of the leg/cable connection on the end-effector, expressed in the body frame
- $\mathbf{p}, {}^W R_P$ Pose of the end-effector w.r.t. the world frame
- $\mathbf{l}_i = \rho_i \mathbf{n}_i$ Vector of the leg/cable in the world frame

For example, for an hexapod platform where the legs are rigidly attached to a plane (i.e., $\mathbf{a}_i$ is fixed), given the pose of the robot we can calculate the joint variable ρ_i from the loop-closure constraint.

Forward Kinematics

Contrarily to serial manipulators, for parallel manipulators the **forward kinematics** are generally more complex.

The **loop-closure equations** are typically highly nonlinear expressions of the pose variables. They form a nonlinear set of equations that **generally admits multiple solutions**.

For example, a Stewart (Gough) platform can have up to 40 solutions

Example: **2P planar parallel robot**

With the same lengths of the legs, we can find multiple poses of the end-effector

